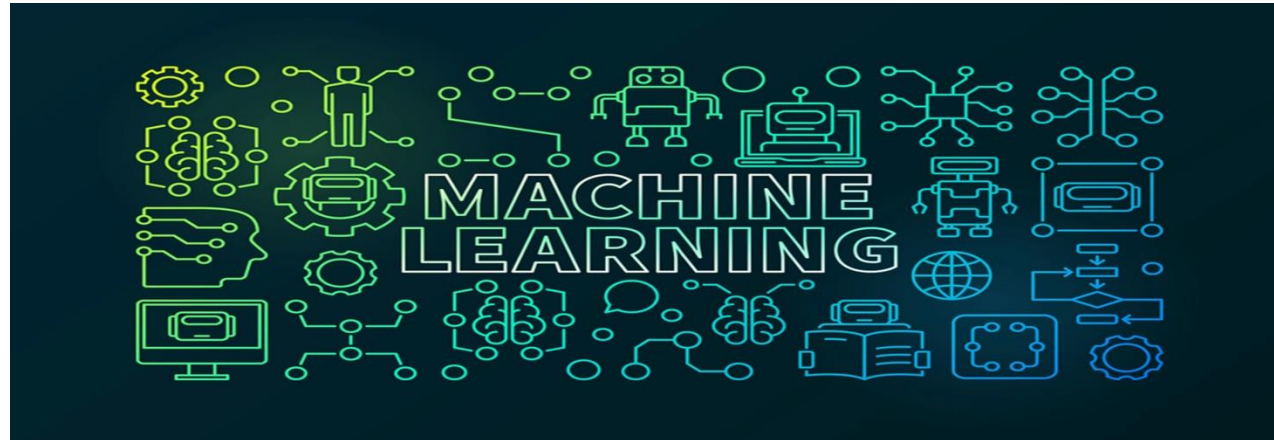




Machine Learning I

Session 3: Linear Regression



Information Technology Institute
Ministry of Communication and Information Technology
Powered by EPITA

Session Outline

1-Recap of previous sessions

2- Introduction to regression

- 2-1 What is regression
- 2-2 Regression use cases
- 2-3 Types of regression

3-What is linear regression

3-1 Theory of the linear regression

3-2 Finding best fit using least squared method

3-3 Checking quality of regression using R-Squared method

4- Implementation of linear regression using Python

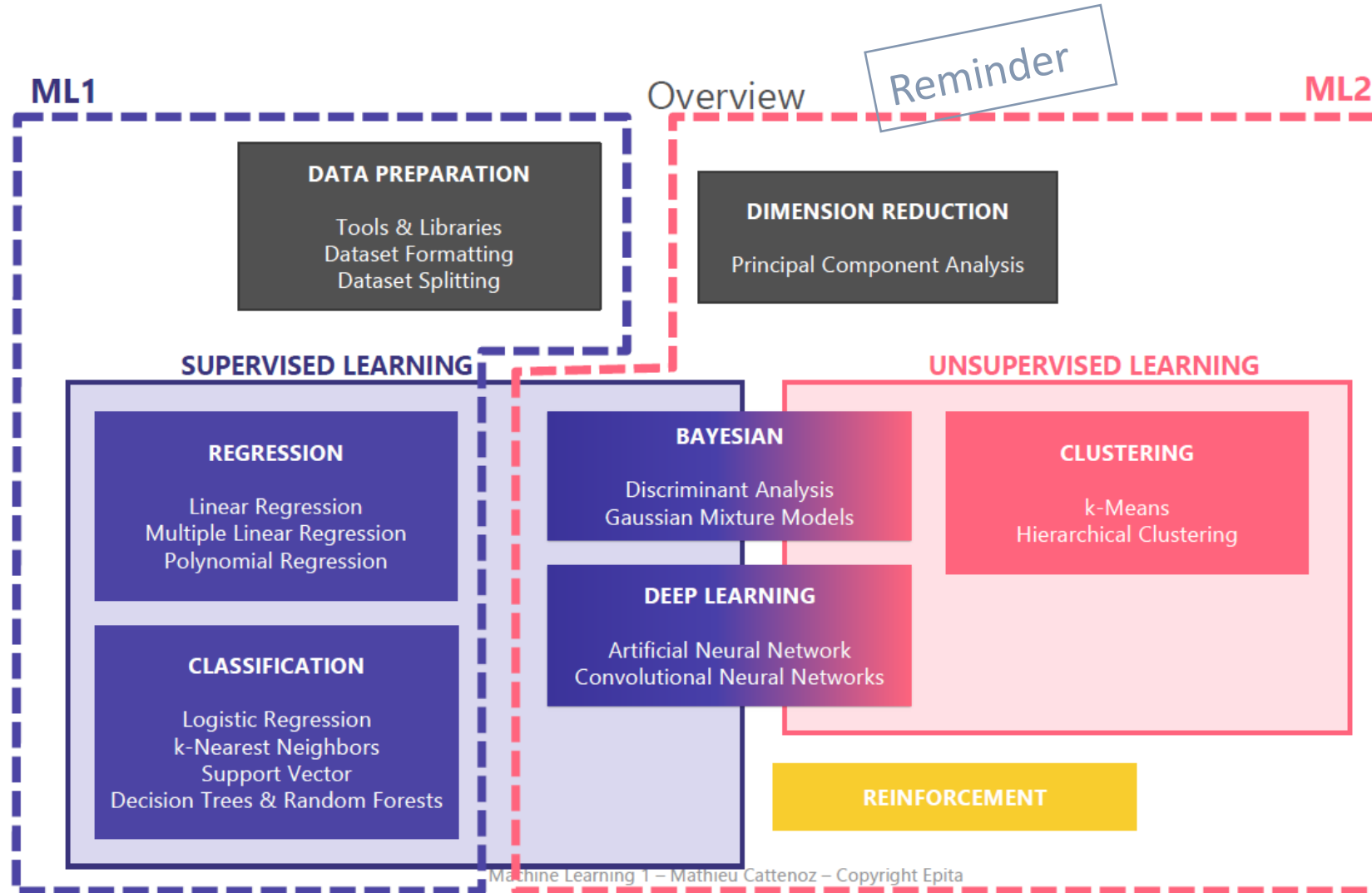
4-1 Linear regression Implementation from scratch

4-2 Linear regression using scikit Lib.

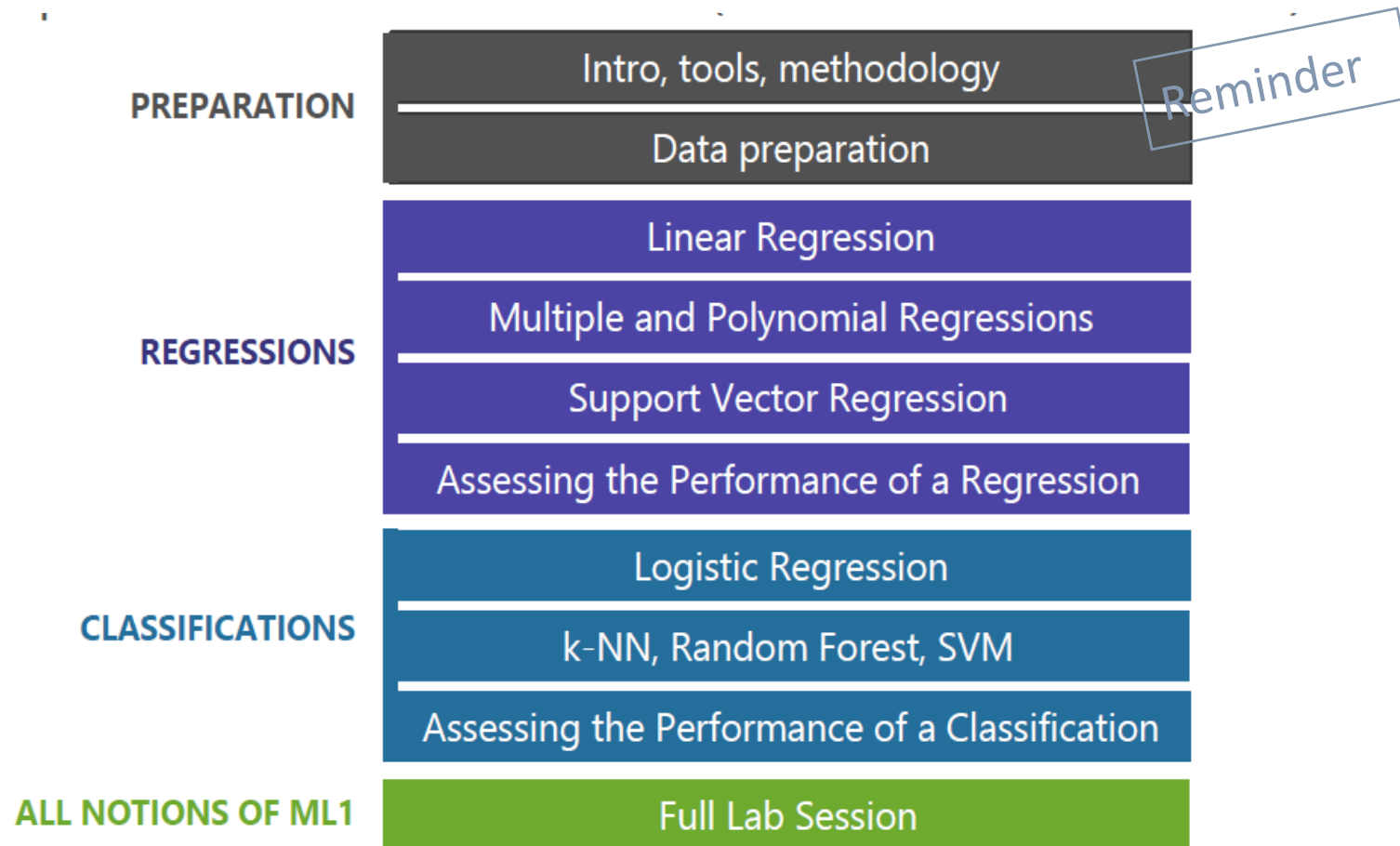
5-Assignment Description

- Assignment #2 Description
- Teacher correction and Q&A.

ML1 & ML2 Overview

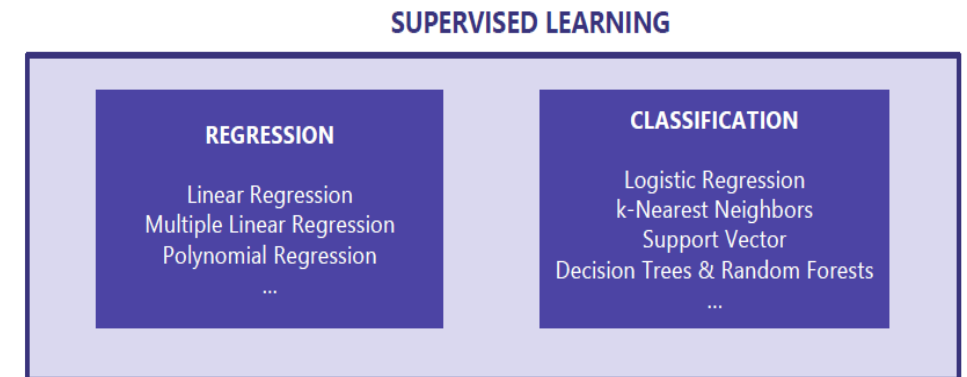
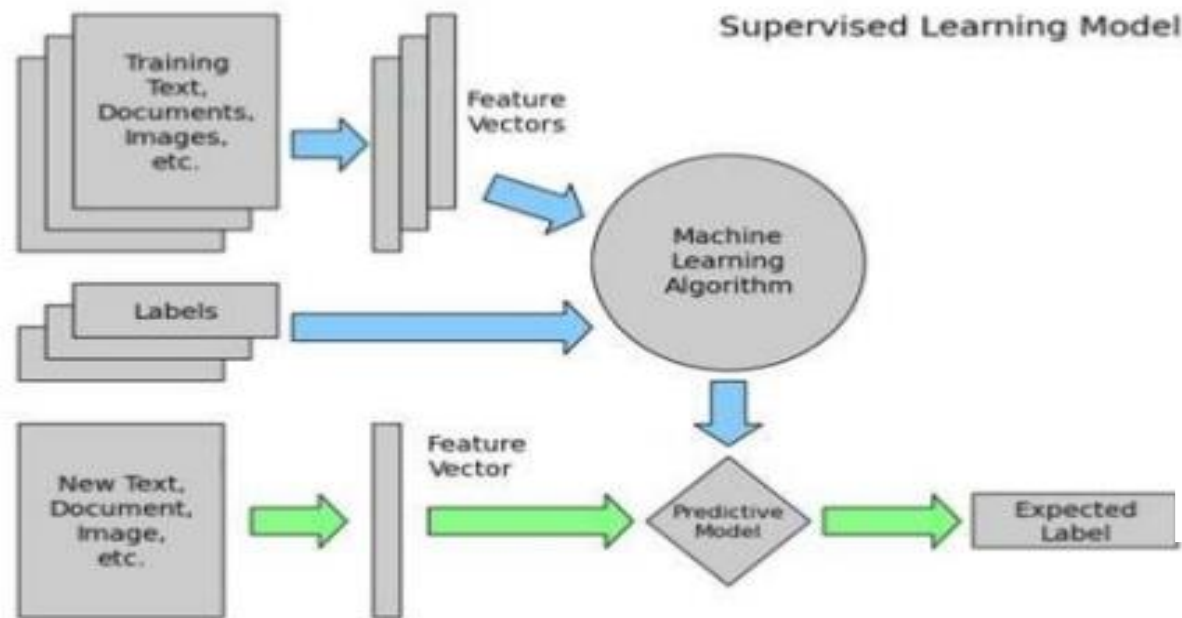


ML1 Structure: (10 sessions of 3 hours):



Applications of supervised learning

- Supervised Learning can deal with two kinds of problems:
 - Classification
 - Regression analysis



Classification Vs. Regression Analysis Supervised Learning

- **Classification algorithms** work only when we have a limited set of possible results. But they can't help when the output has to be a numerical value we are trying to predict.
- classification and regression analysis algorithms are very similar and differ only in the possible results they can produce.
- **For classification the output(s) is nominal** categorical
- **In regression the output is continuous**

When Do You Need Regression?

- Typically, you need regression to answer whether and how some phenomenon influences the other or **how several variables are related.**
 - For example, you can use it to determine *if* and *to what extent* the experience or gender impact salaries.
- Regression is also useful when you want **to forecast a response** using a new set of predictors.
 - For example, you could try to predict electricity consumption of a household for the next hour given the outdoor temperature, time of day, and number of residents in that household.

Use-cases for regression analysis

- Forecasting stock prices
- Forecasting currency exchange rates
- Estimating real estate prices
- Estimating used car prices
- Predicting energy consumption for buildings
- Predicting sales/profit estimates

Types of regression

1. Linear Regression

Simple Linear

Multiple Linear

2. Logistic Regression

3. Polynomial Regression

4. Ridge Regression

5. Lasso Regression

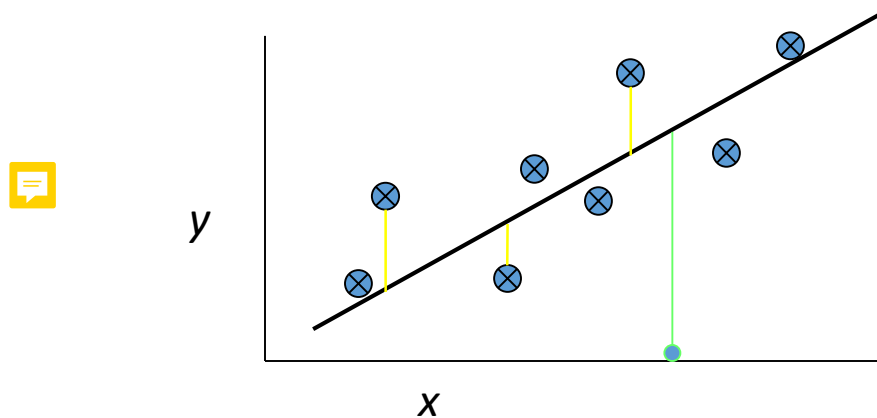
6. Poisson Regression

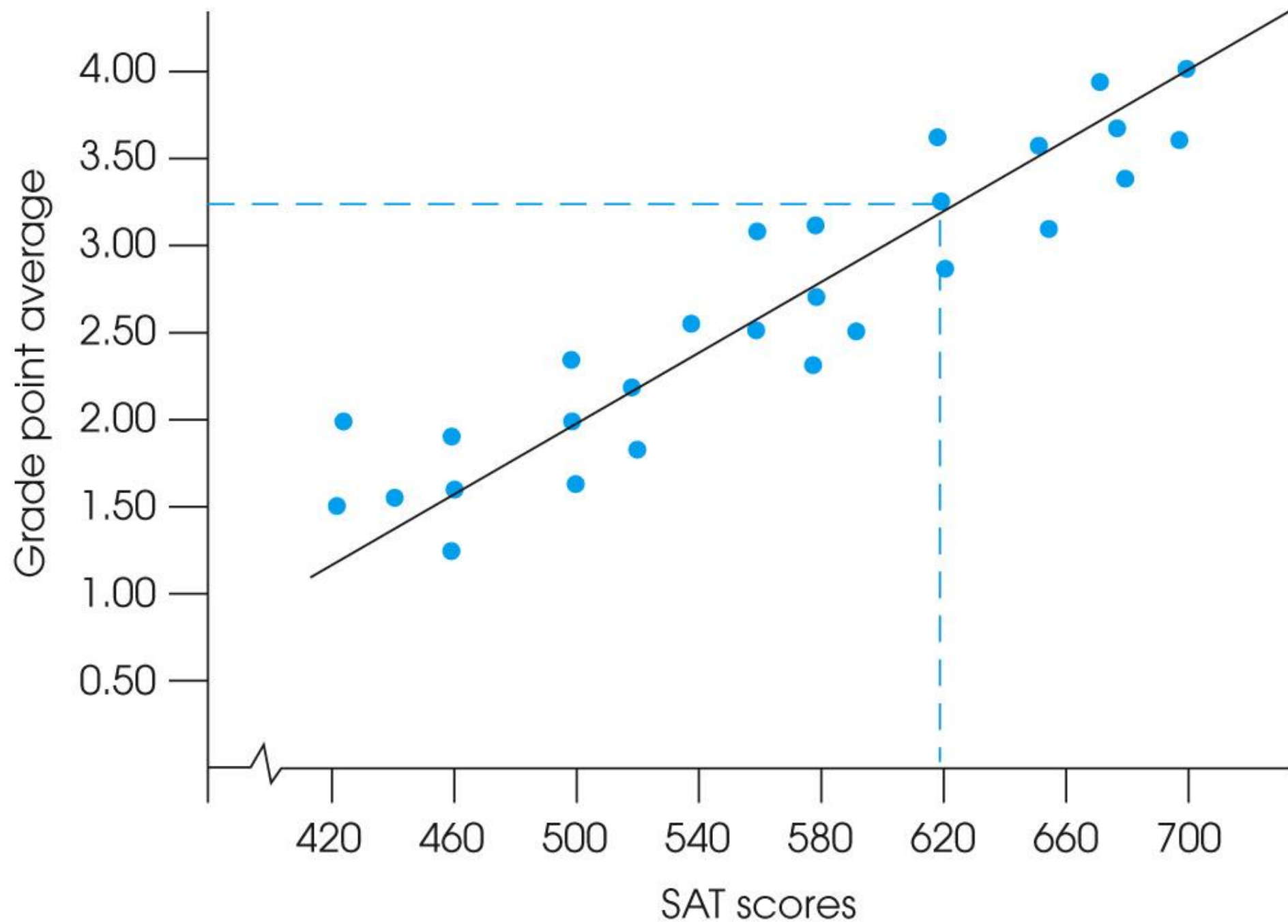
When to use Linear Regression

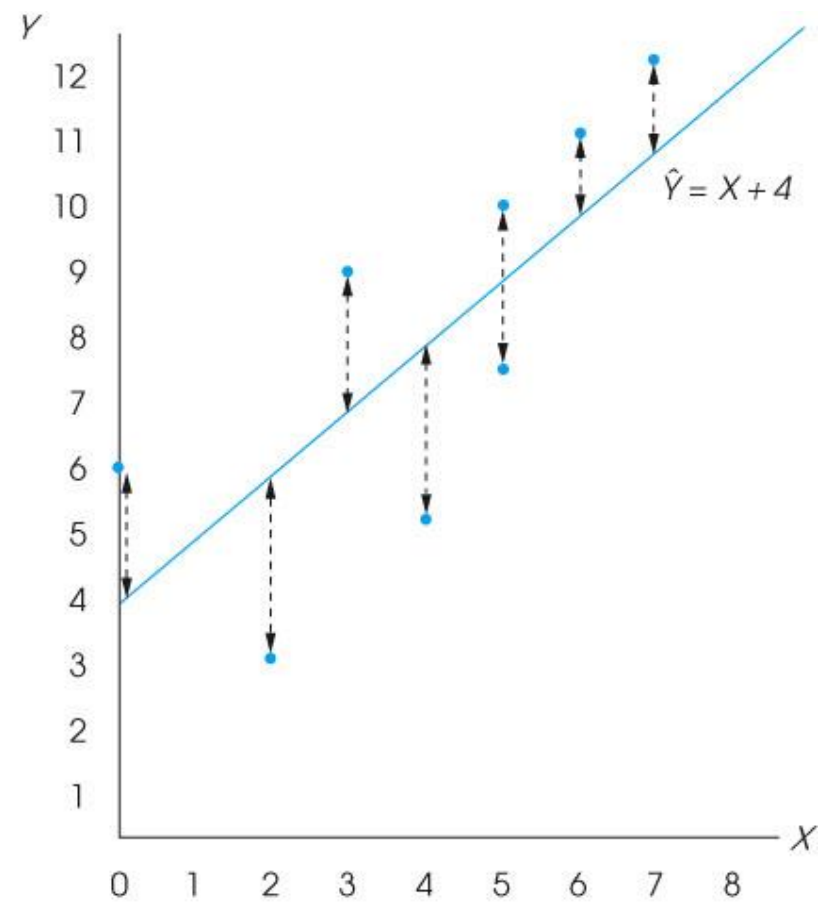
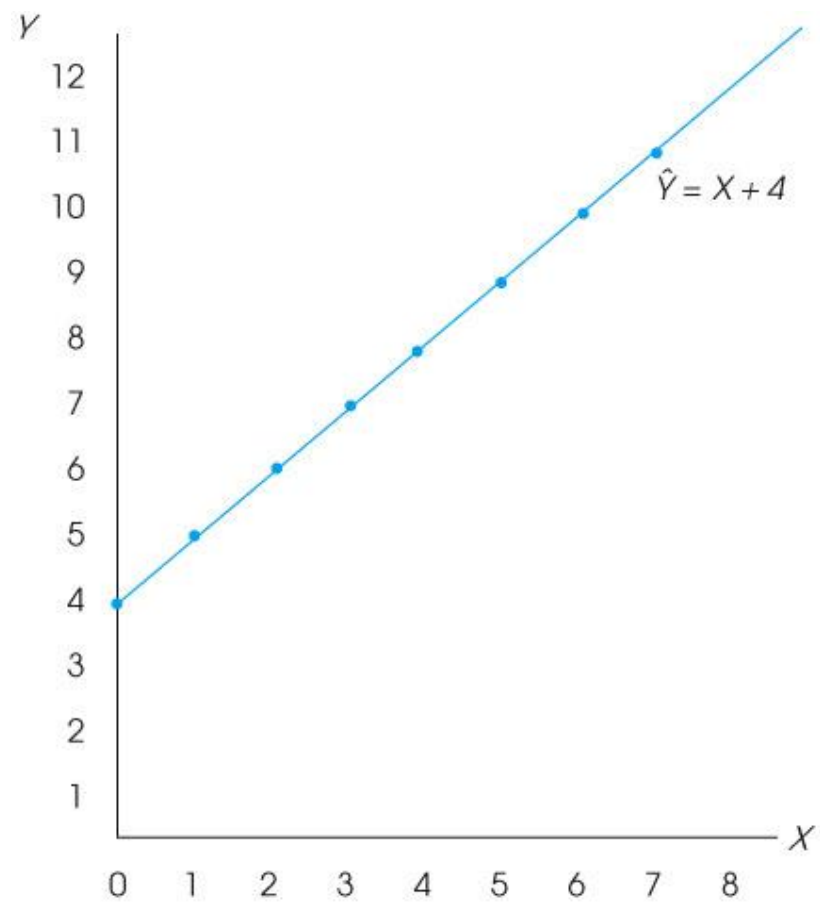
- The relationship between the predictor variable(s) and the response variable is reasonably linear.
- The response variable is a continuous numeric variable.
 - **Example:** A retail company may fit a linear regression model using advertising spend to predict total sales.
 - Since the relationship between these two variables is likely linear (more money spent on advertising generally leads to an increase in sales) and the response variable (total sales) is a continuous numeric variable, it makes sense to fit a linear regression model.

Linear Regression

- Many models could be used – Simplest is linear regression
 - Fit data with the best hyper-plane which "goes through" the points
 - For each point the difference between the predicted point and the actual observation is the *residue*







Introduction to Linear Regression (cont.)

- How well a set of data points fits a straight line can be measured by calculating the distance between the data points and the line.
- The total error between the data points and the line is obtained by squaring each distance and then summing the squared values.
- The regression equation is designed to produce the minimum sum of squared errors.

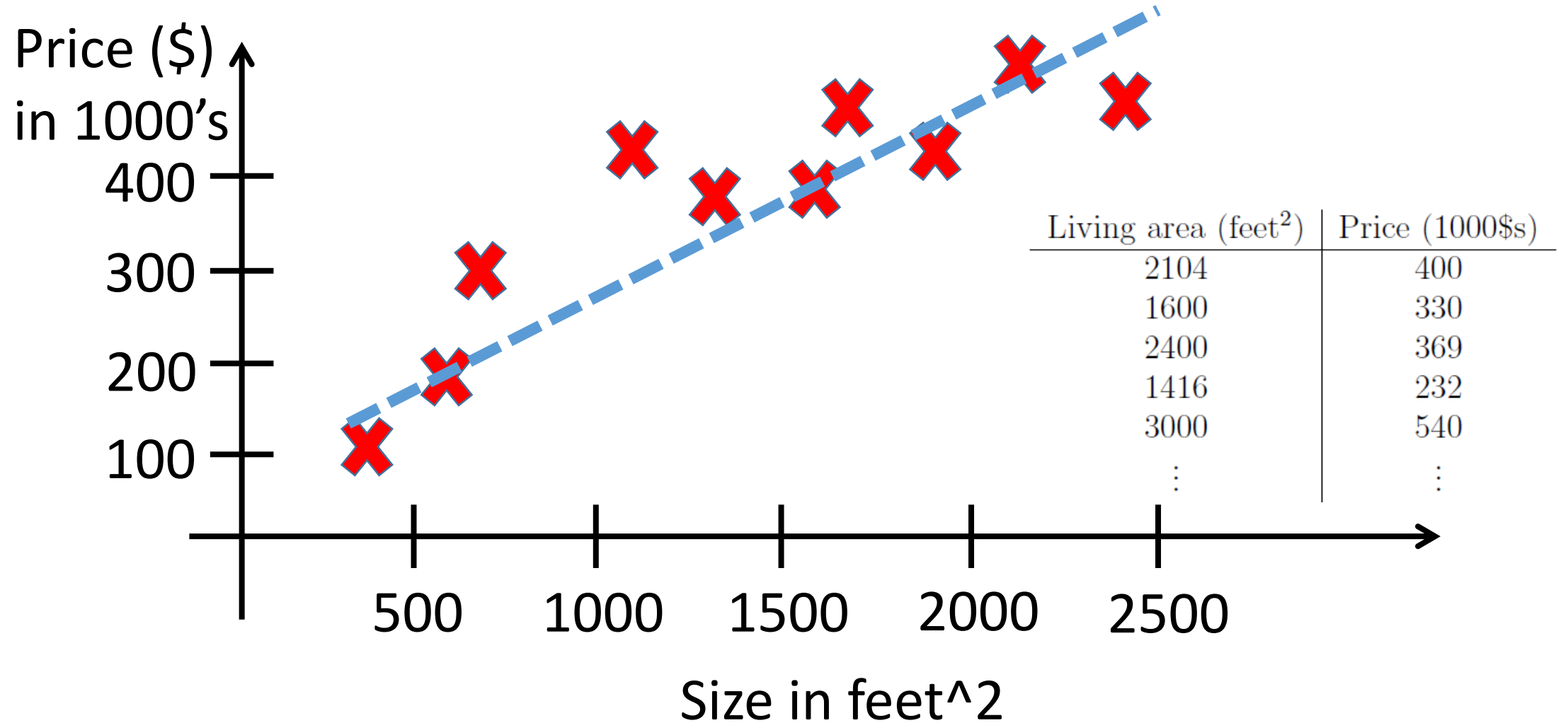
Regression Analysis Problems in Supervised Learning

- To apply a regression analysis algorithm data scientist should go through the same process as we described before.
 - It should collect data which contains input parameters and correct answers (labels).
 - This data is fed to a regression analysis algorithm and it produces a trained model.
 - After getting the model, we can use it for predicting new values using only input parameters.

Variable's Roles

- Dependent variable (y) –Y axis
 - Value we want to predict or explain
 - Depend on something else
- Independent variable(s) (x) X-axis
 - Other variable depend on it

House pricing prediction



Regression

real-valued output

Training set

Learning Algorithm

x

h

y

Size of house

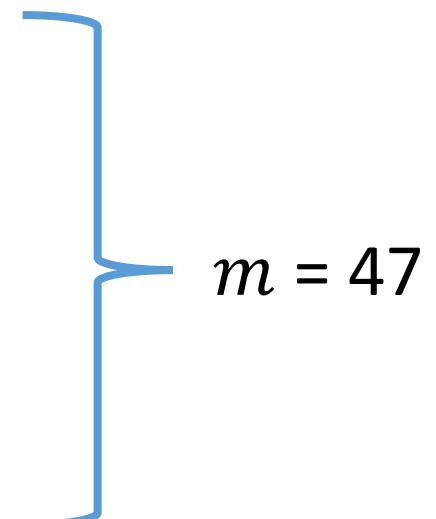
Hypothesis

Estimated price



Training set

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...



- Notation:

- m = Number of training examples
- x = Input variable / features
- y = Output variable / target variable
- (x, y) = One training example
- $(x^{(i)}, y^{(i)}) = i^{th}$ training example

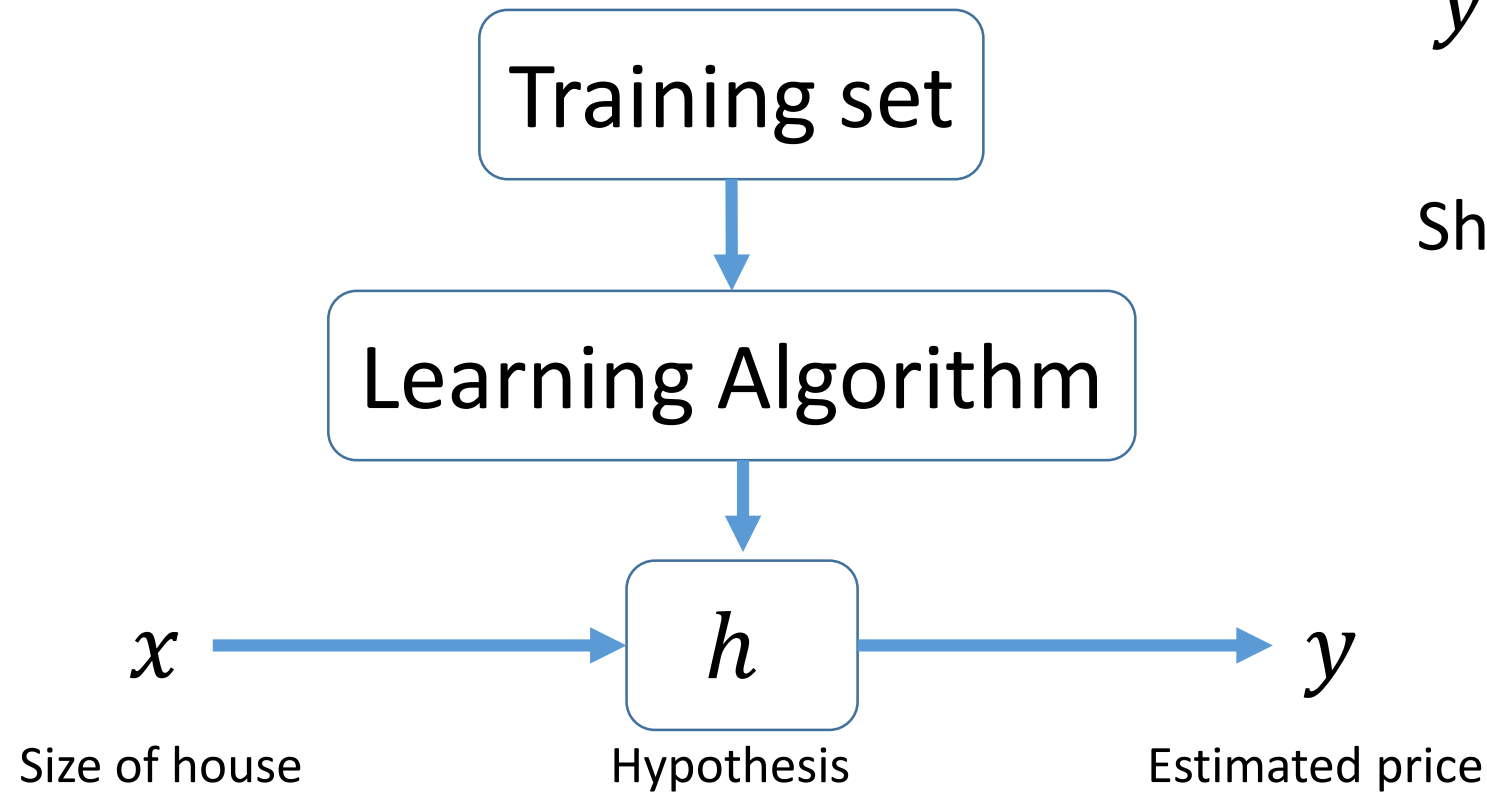
Examples:

$$x^{(1)} = 2104$$

$$x^{(2)} = 1416$$

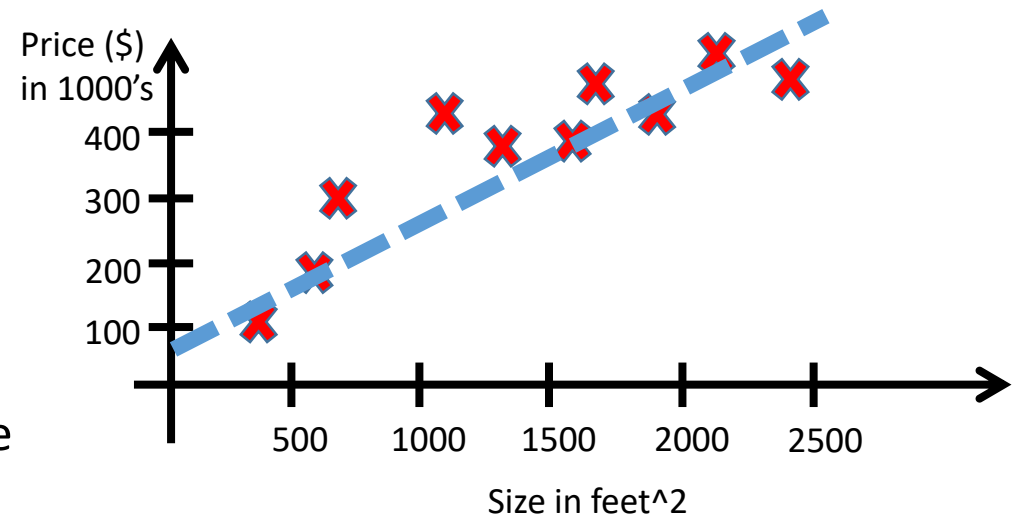
$$y^{(1)} = 460$$

Model representation



$$y = h_{\theta}(x) = \theta_0 + \theta_1 x$$

Shorthand $h(x)$



Univariate linear regression

Linear Regression

1 • Model representation

2 • Cost function

3 • Gradient descent

Training set

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

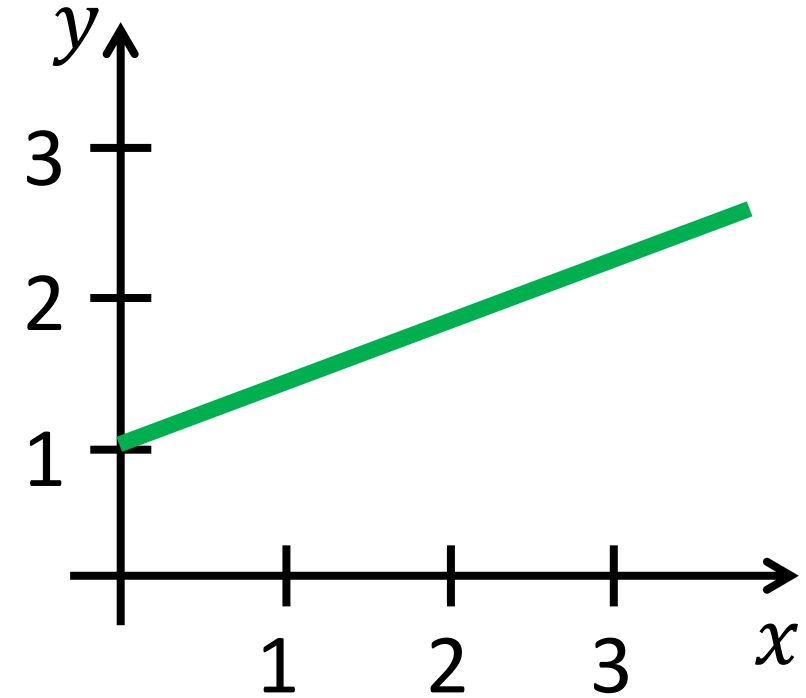
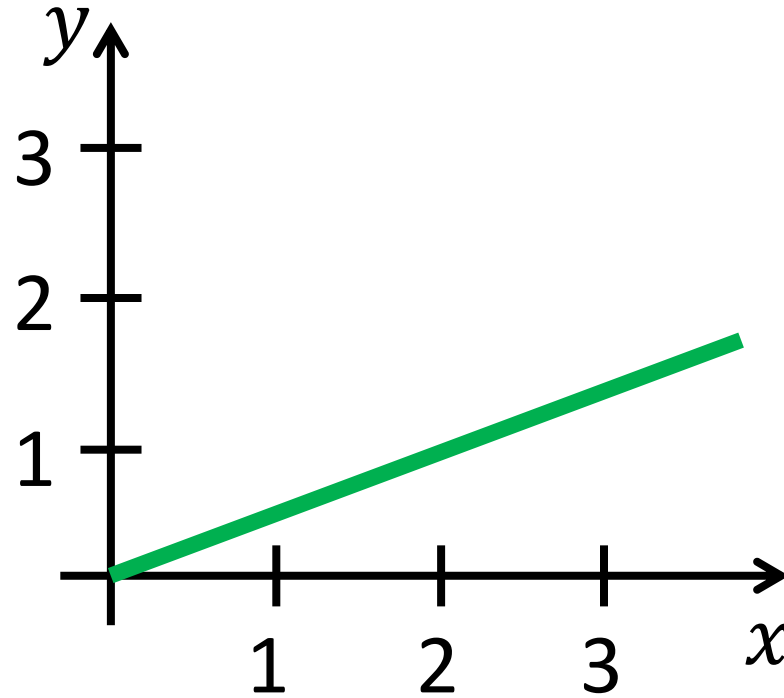
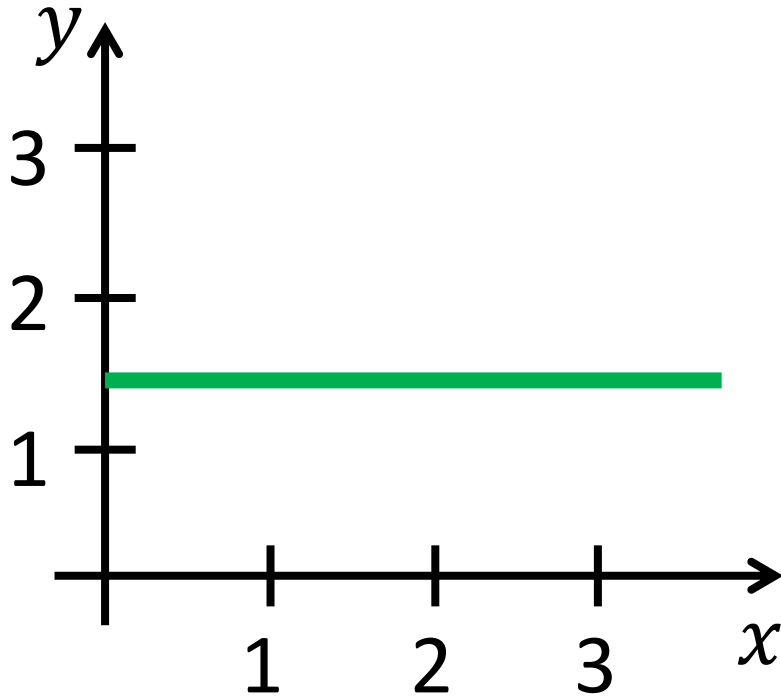
} $m = 47$

• Hypothesis $h_{\theta}(x) = \theta_0 + \theta_1 x$

θ_0, θ_1 : parameters/weights

How to choose θ_i 's?

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



Y-intercept $\theta_0 = 1.5$

Slope $\theta_1 = 0$

$\theta_0 = 0$

$\theta_1 = 0.5$

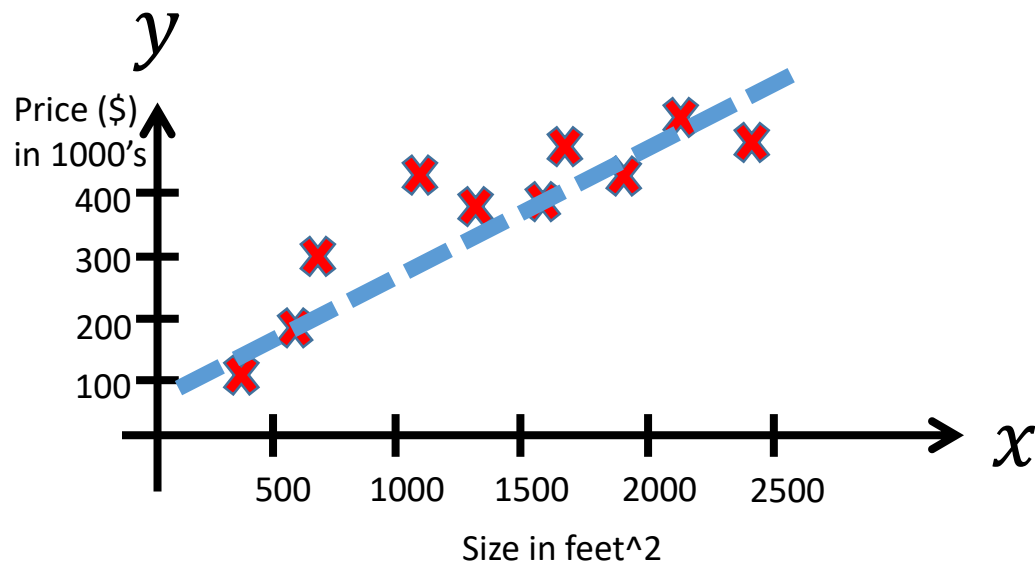
$\theta_0 = 1$

$\theta_1 = 0.5$

Cost function

- Idea:

Choose θ_0, θ_1 so that $h_\theta(x)$ is close to y for our training example (x, y)



minimize $\frac{1}{2m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)})^2$
 θ_0, θ_1

$h_\theta(x^{(i)}) = \theta_0 + \theta_1 x^{(i)}$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)})^2$$

minimize $J(\theta_0, \theta_1)$ Cost function
 θ_0, θ_1

Simplified

- **Hypothesis:**

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



- **Hypothesis:**

$$h_{\theta}(x) = \theta_1 x$$

$$\theta_0 = 0$$

- **Parameters:**

$$\theta_0, \theta_1$$



- **Parameters:**

$$\theta_1$$

- **Cost function:**

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$



- **Cost function:**

$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- **Goal:**

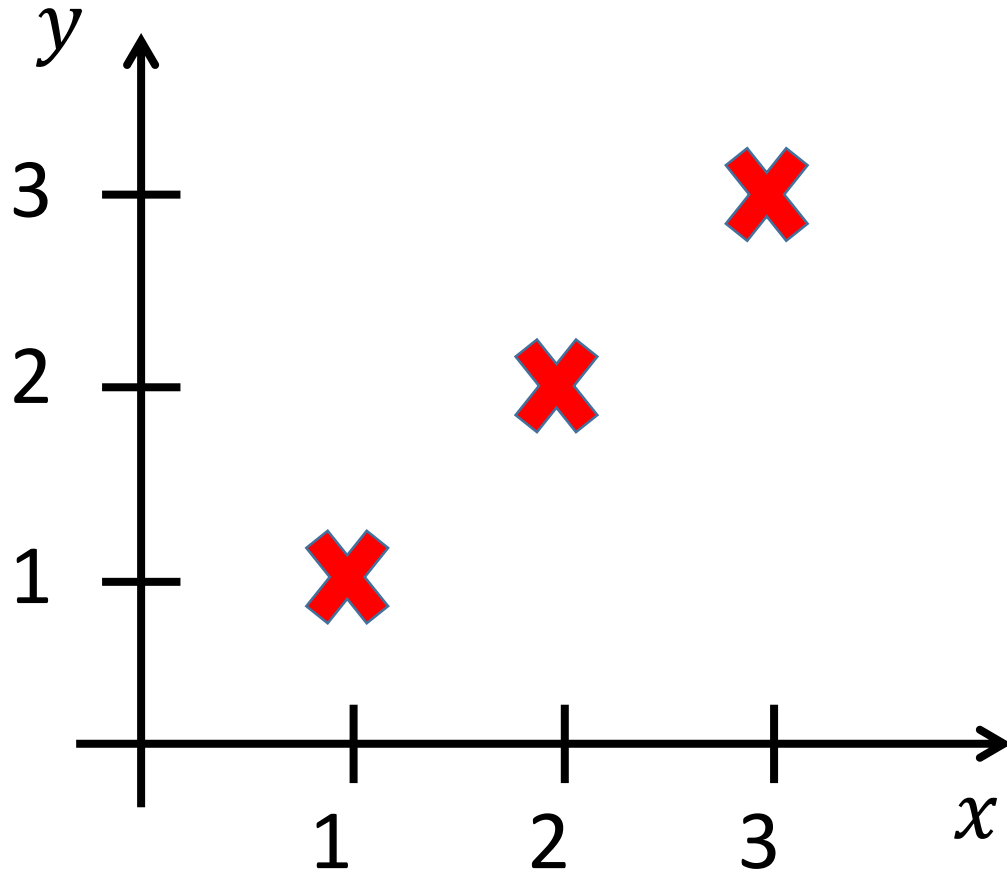
$$\underset{\theta_0, \theta_1}{\text{minimize}} J(\theta_0, \theta_1)$$



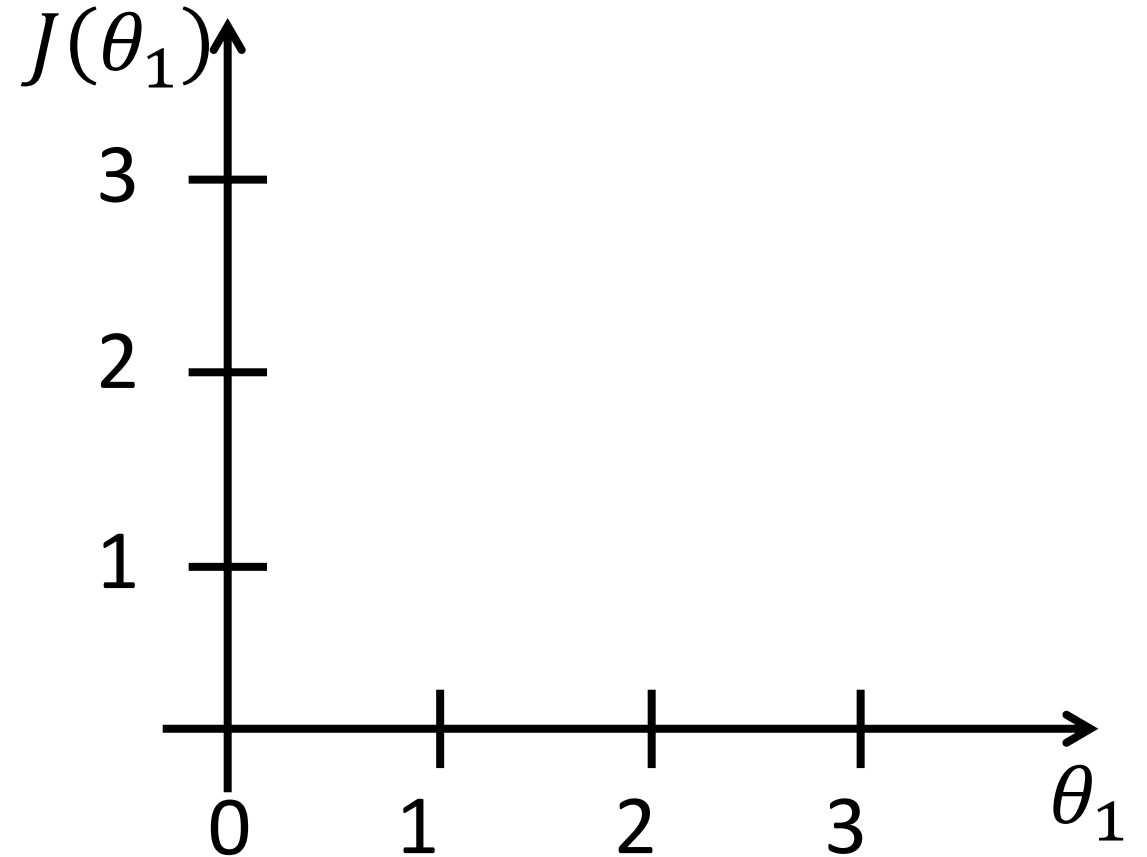
- **Goal:**

$$\underset{\theta_0, \theta_1}{\text{minimize}} J(\theta_1)$$

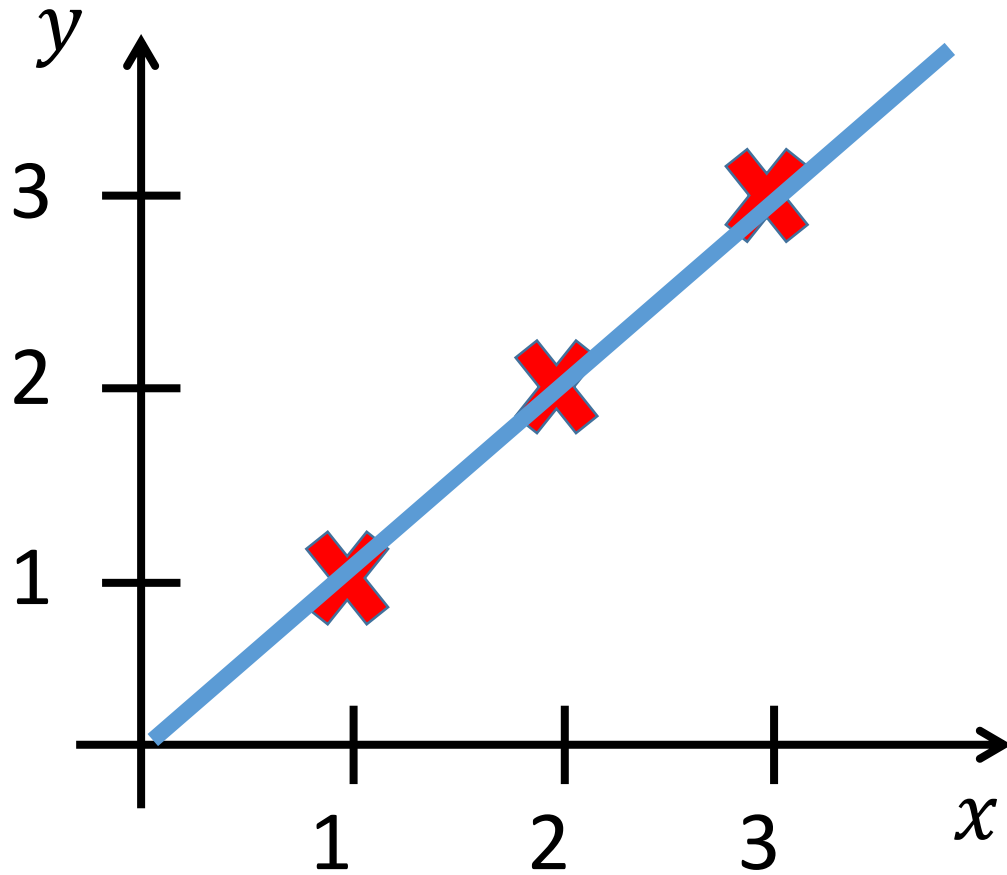
$h_{\theta}(x)$, function of x



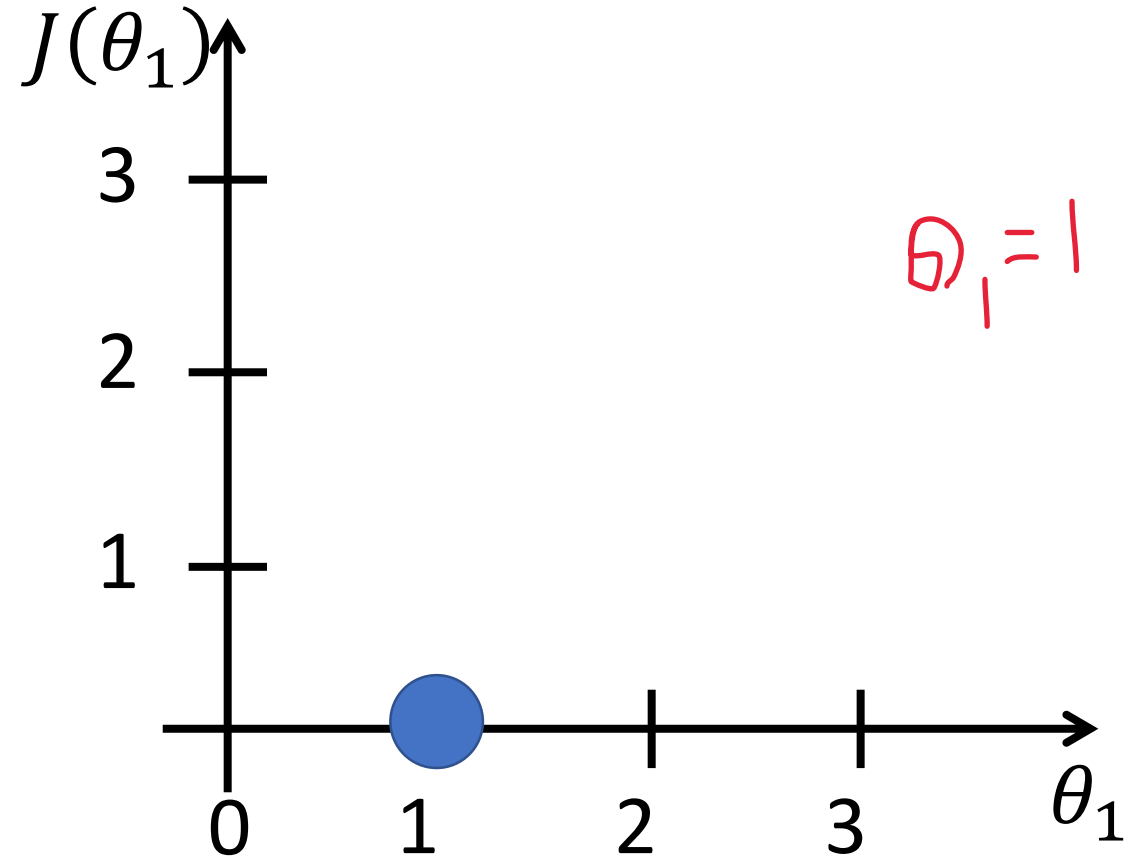
$J(\theta_1)$, function of θ_1



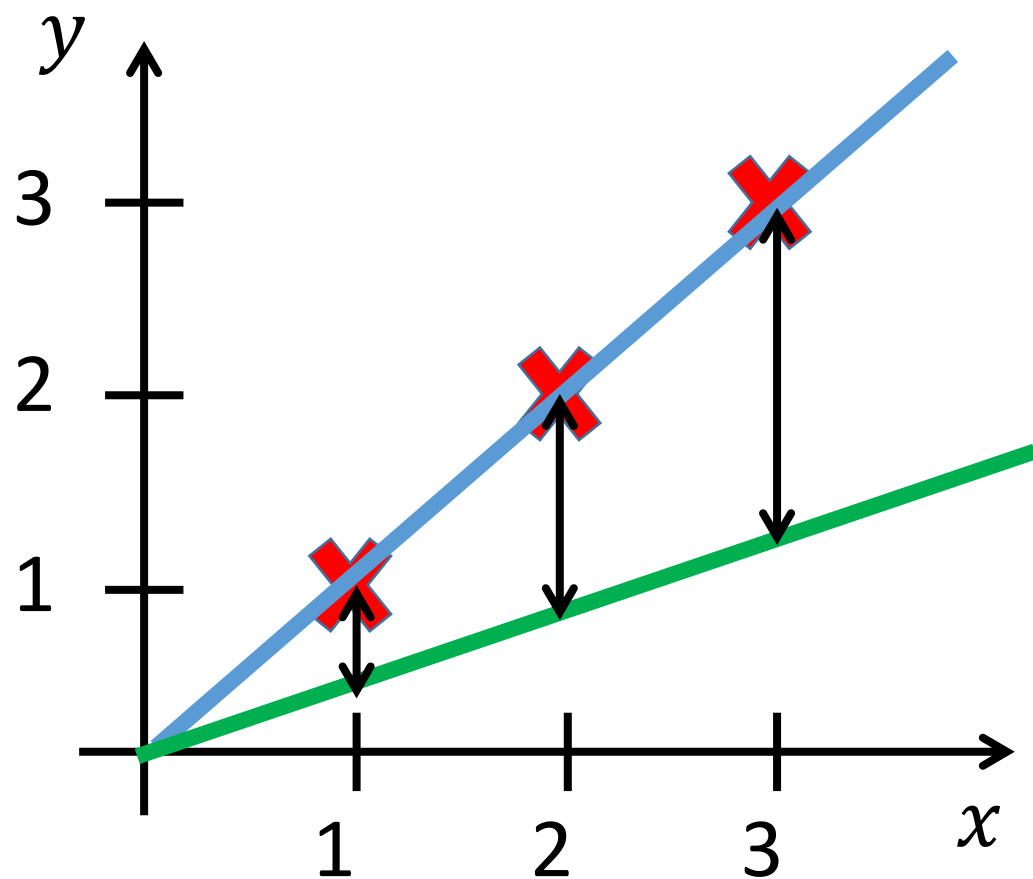
$h_{\theta}(x)$, function of x



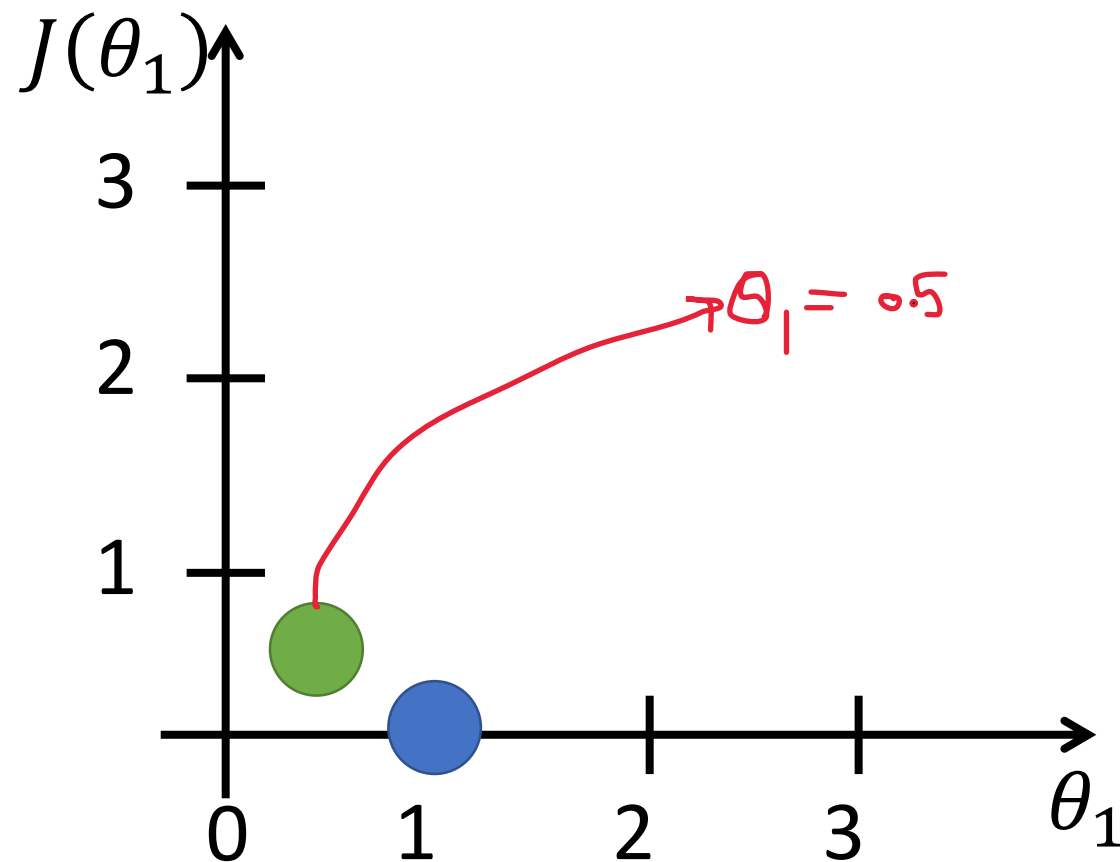
$J(\theta_1)$, function of θ_1



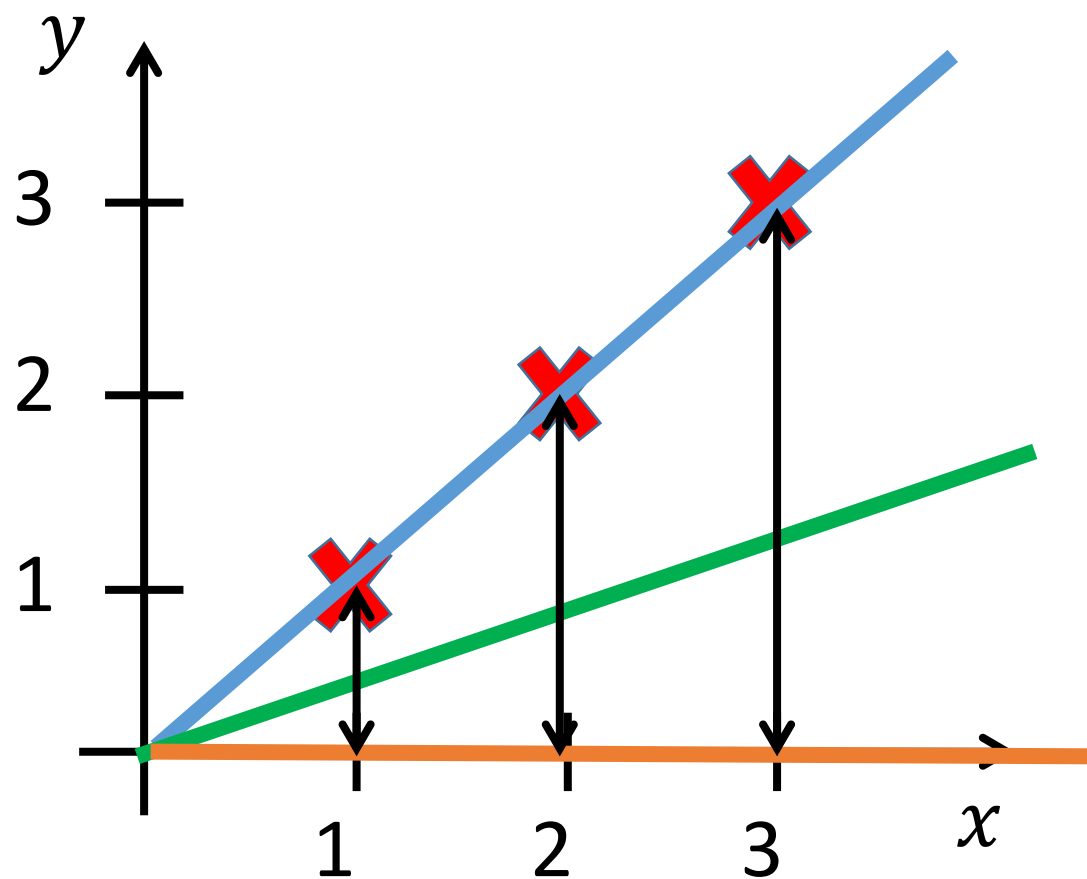
$h_{\theta}(x)$, function of x



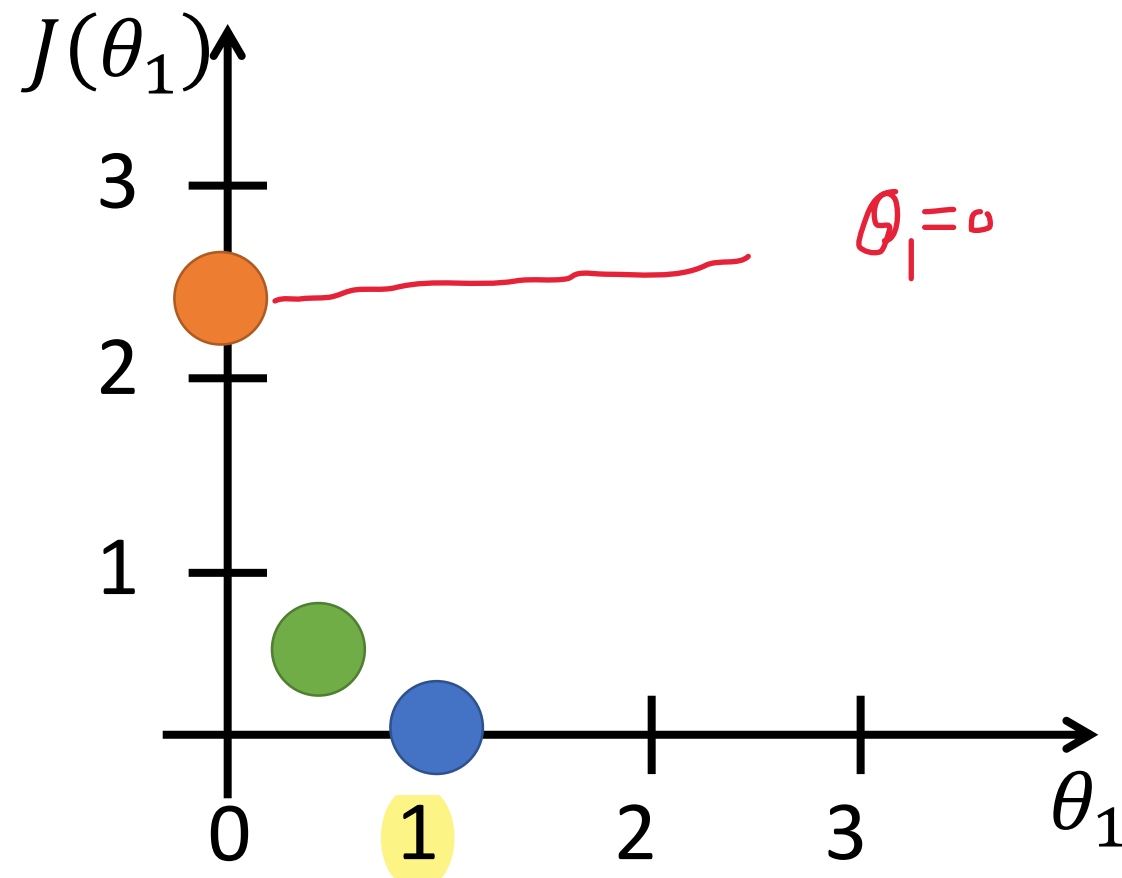
$J(\theta_1)$, function of θ_1



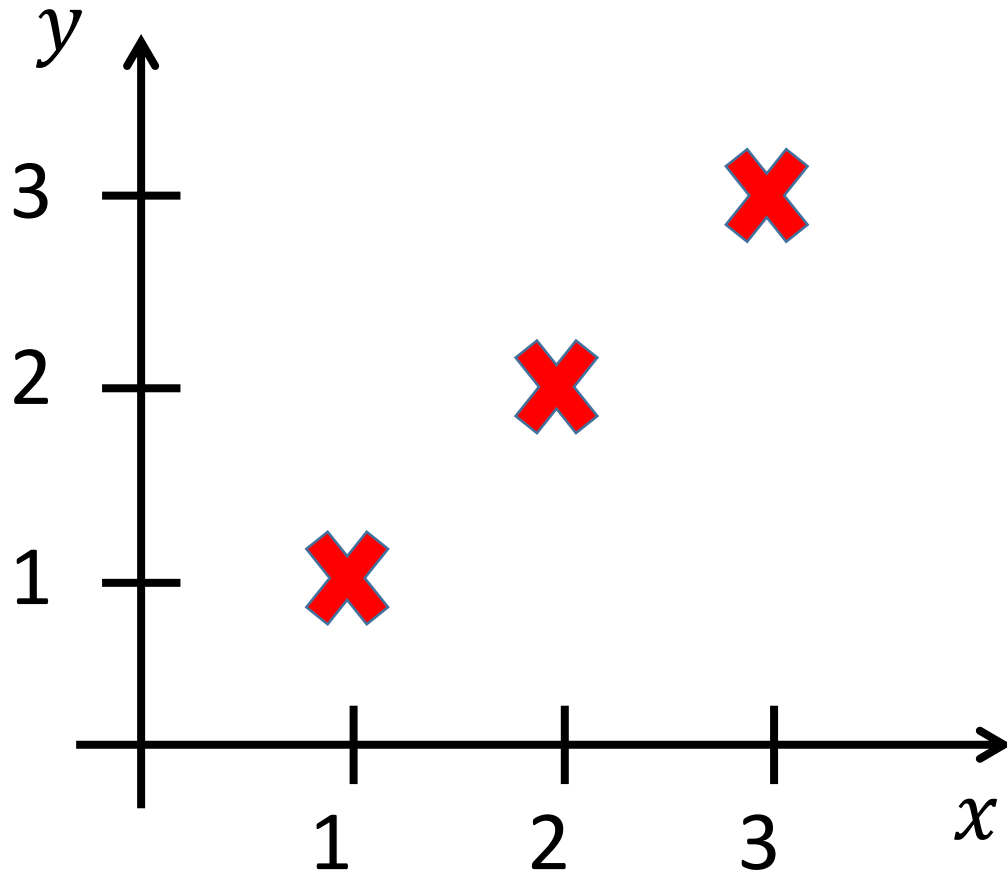
$h_{\theta}(x)$, function of x



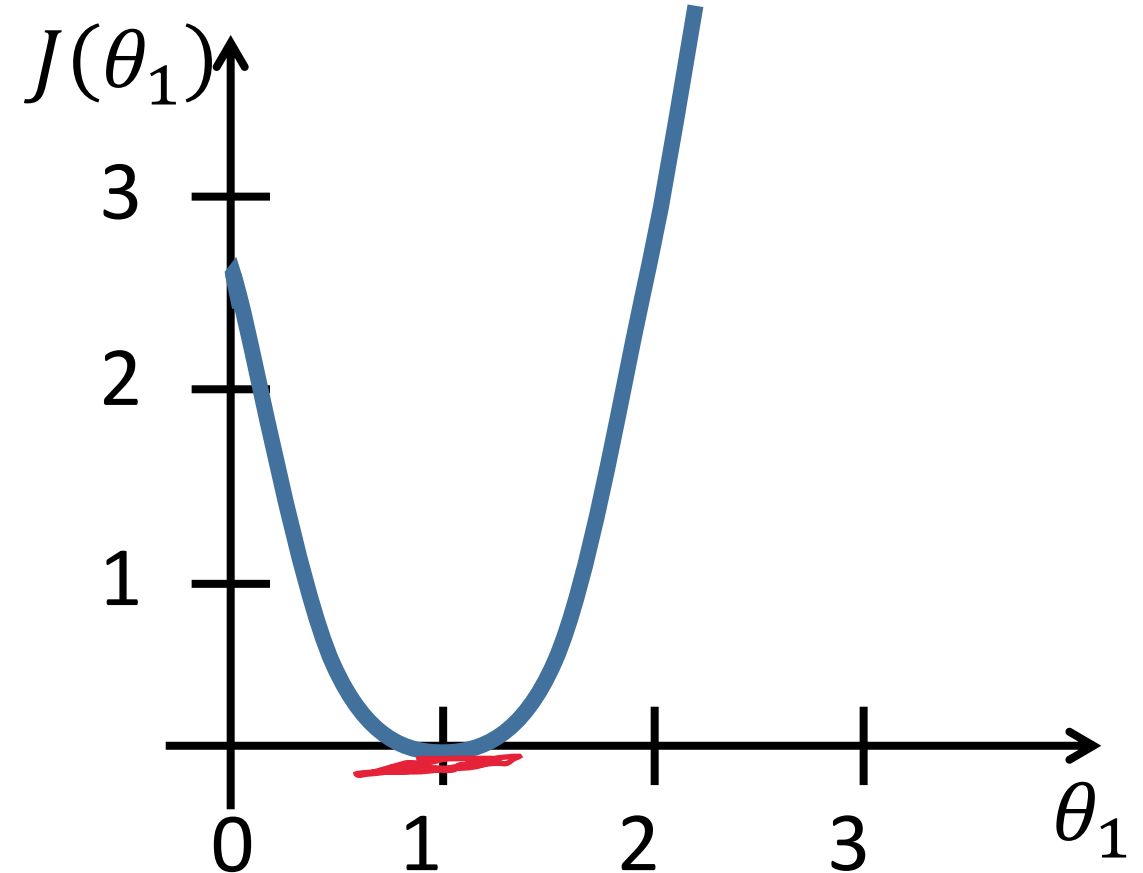
$J(\theta_1)$, function of θ_1



$h_{\theta}(x)$, function of x

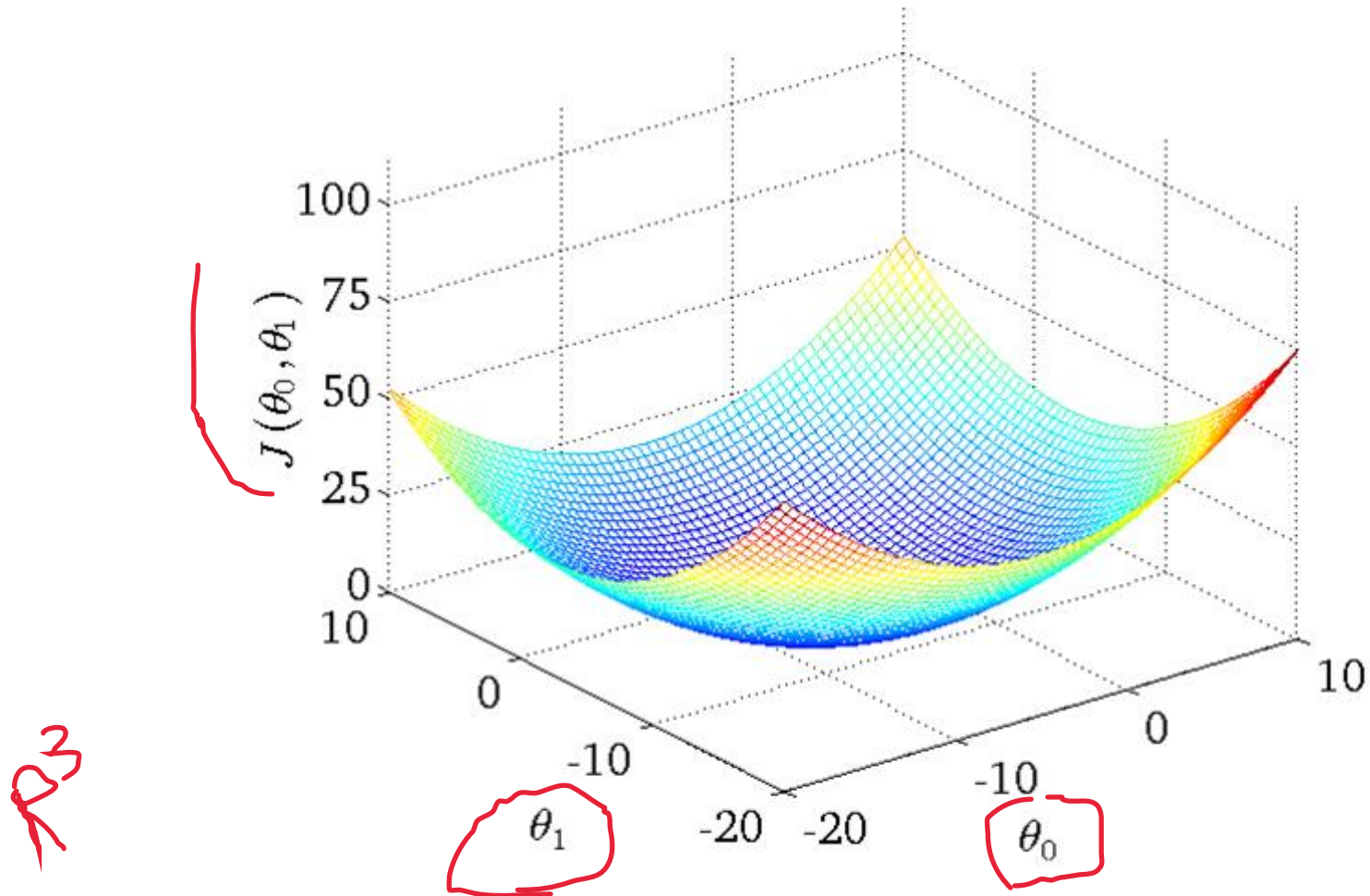


$J(\theta_1)$, function of θ_1

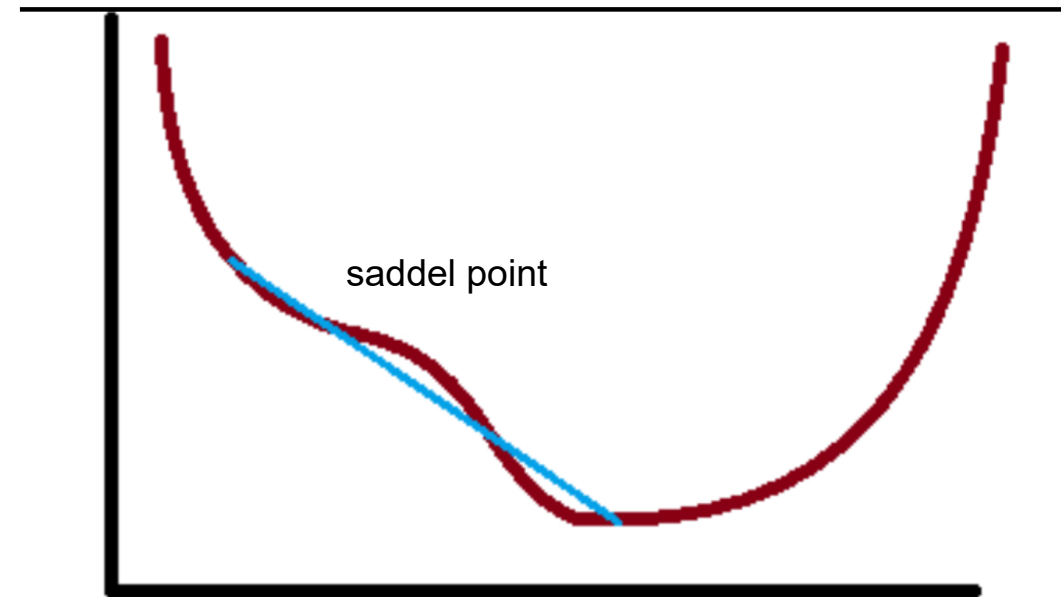
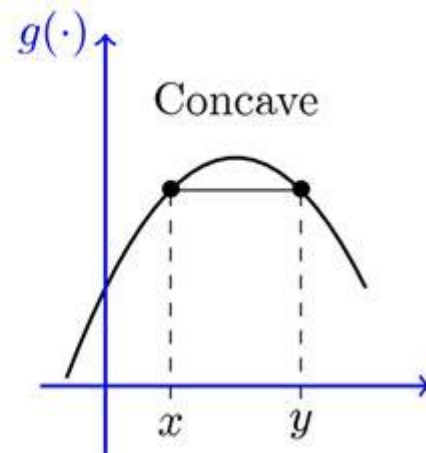
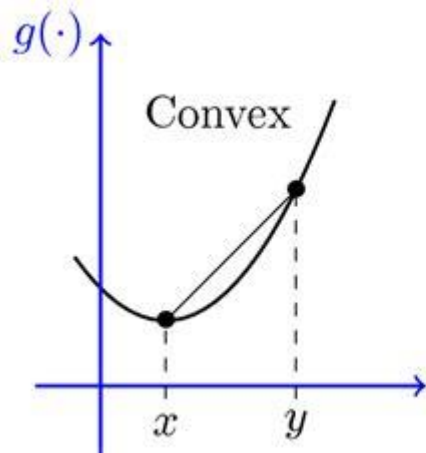


- **Hypothesis:** $h_{\theta}(x) = \theta_0 + \theta_1 x$
- **Parameters:** θ_0, θ_1
- **Cost function:** $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$
- **Goal:** minimize $J(\theta_0, \theta_1)$
 θ_0, θ_1

Cost function



Gradient decent and non-convex functions



SSE and Linear Regression



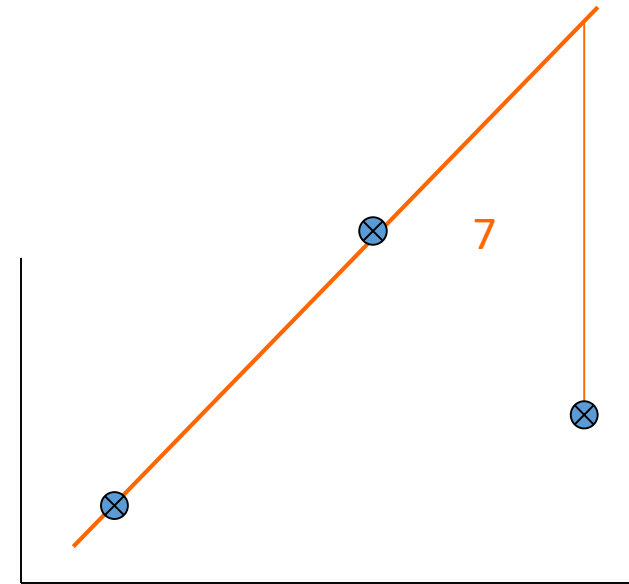
- SSE chooses to square the difference of the predicted vs actual. Why square?
- Don't want residues to cancel each other
- Could use absolute or other distances to solve problem
 $\sum |predicted_i - actual_i|$: L1 vs L2
- SSE leads to a parabolic error surface which is good for gradient descent
- Which line would least squares choose?
 - There is always one “best” fit



In summary, SSE is a key metric used to assess the goodness of fit of a linear regression model to the observed data. By minimizing the SSE, the model aims to find the optimal set of coefficients that best represents the relationship between the dependent and independent variables.

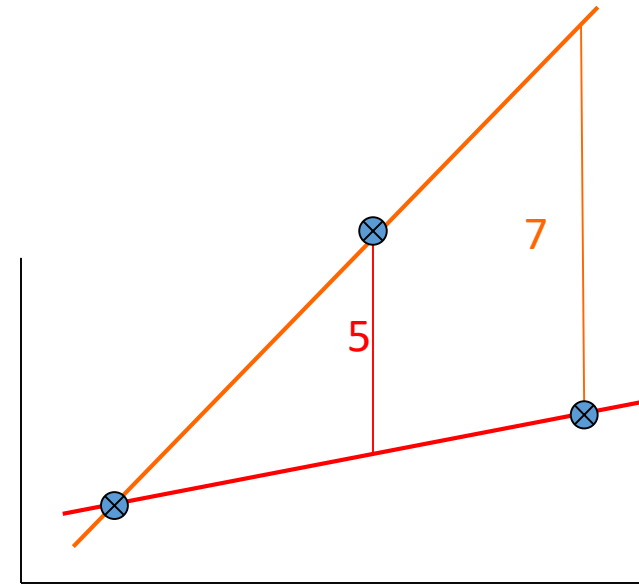
SSE and Linear Regression

- SSE leads to a parabolic error surface which is good for gradient descent
- Which line would least squares choose?
 - There is always one “best” fit



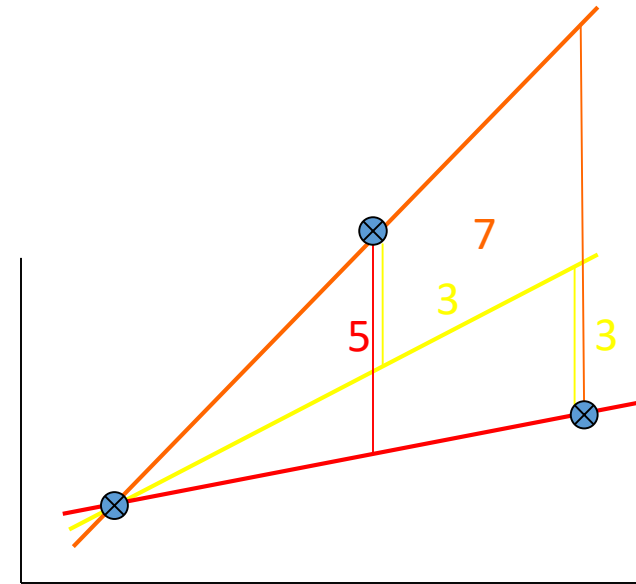
SSE and Linear Regression

- SSE leads to a parabolic error surface which is good for gradient descent
- Which line would least squares choose?
 - There is always one “best” fit



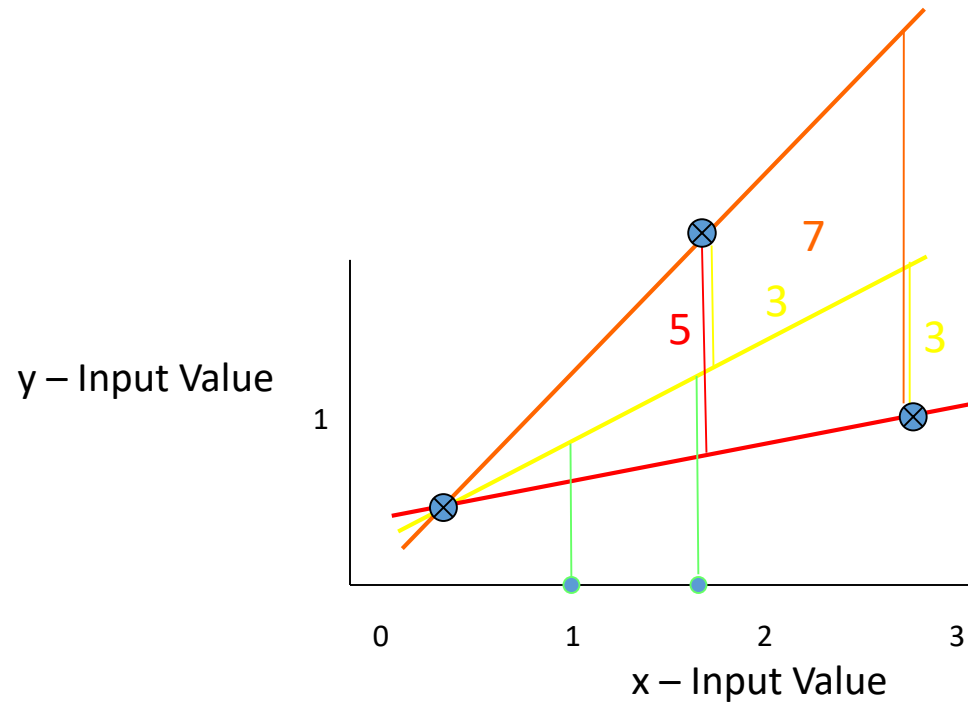
SSE and Linear Regression

- SSE leads to a parabolic error surface which is good for gradient descent
- Which line would least squares choose?
 - There is always one “best” fit
- Note that the squared error causes the model to be more highly influenced by outliers
 - Though best fit assuming Gaussian noise error from true surface



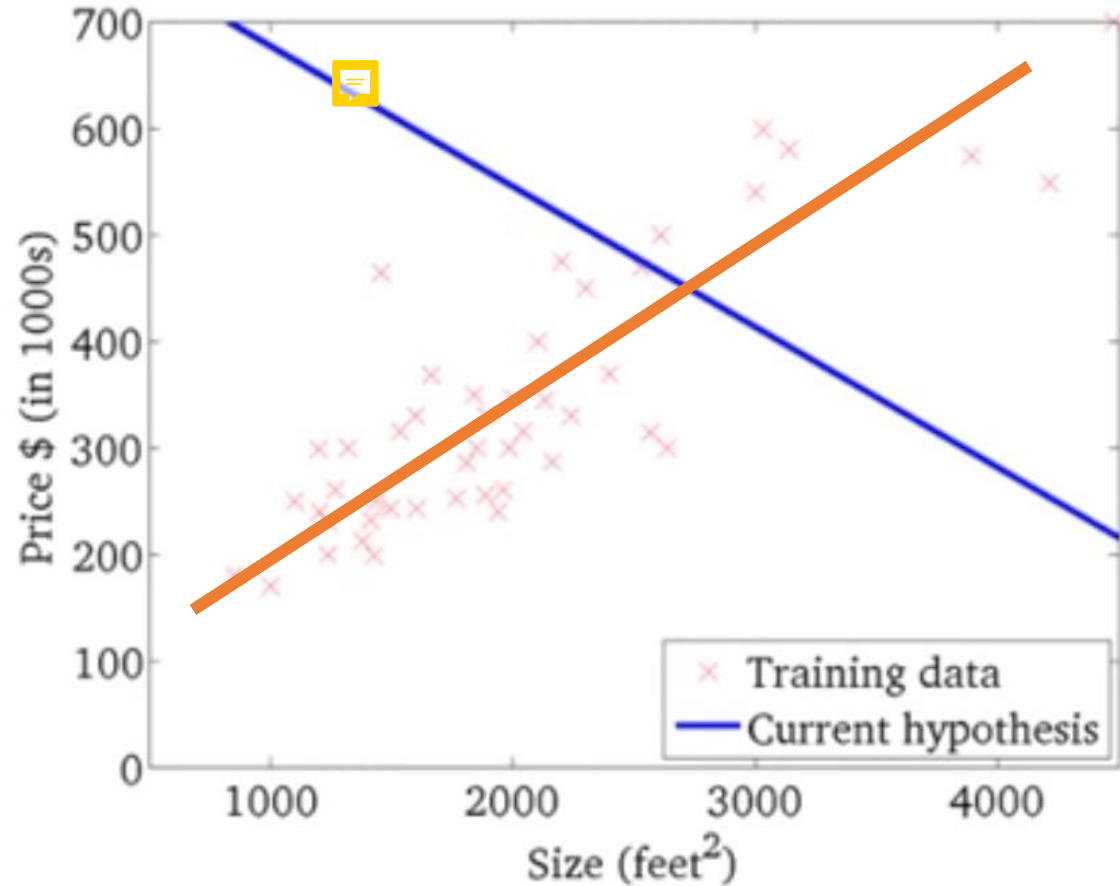
SSE and Linear Regression Generalization

- In generalization all x values map to a y value on the chosen regression line



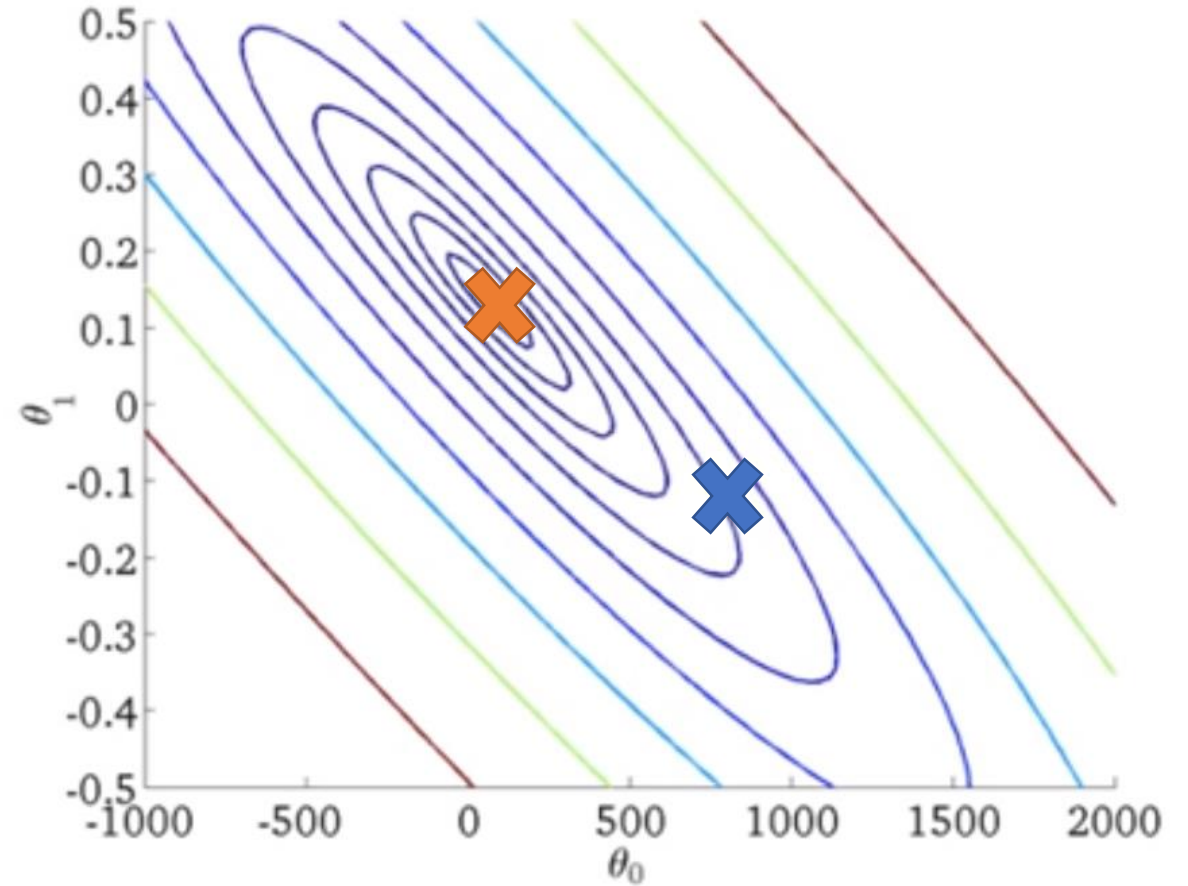
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

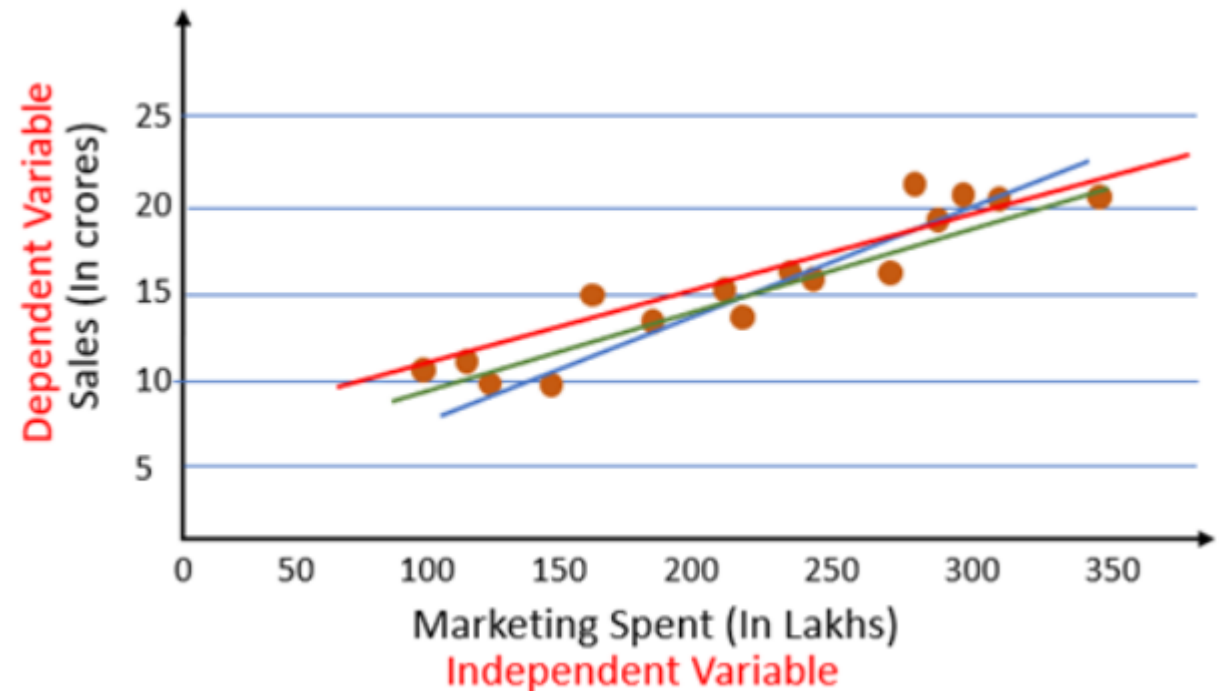
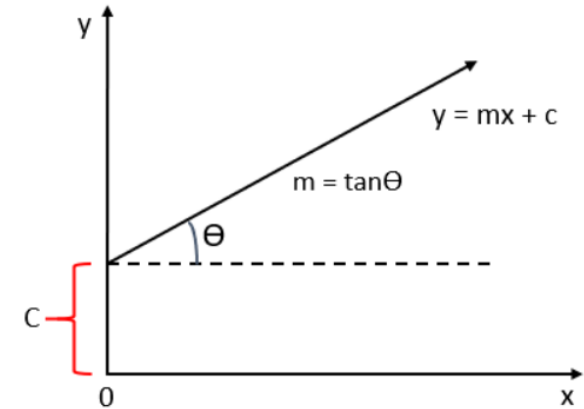
(function of the parameters θ_0, θ_1)



How do we find good θ_0, θ_1 that minimize $J(\theta_0, \theta_1)$?

Gradient Descent in Linear Regression

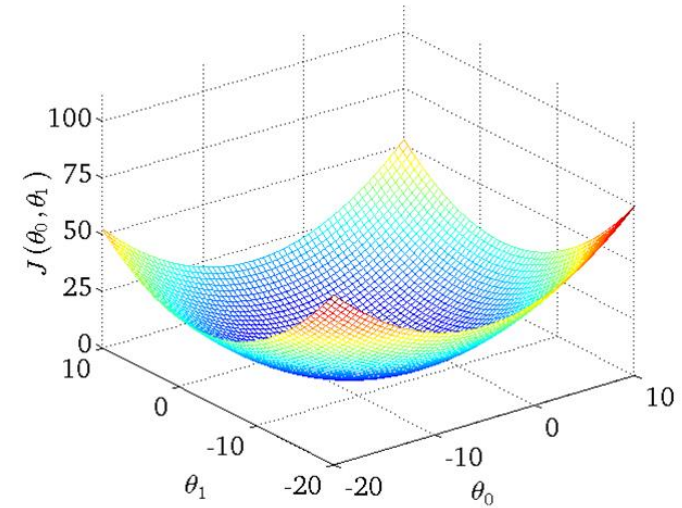
Gradient Descent is an algorithm that finds the best-fit line for a given training dataset in a smaller number of iterations.



Gradient descent

Have some function $J(\theta_0, \theta_1)$

Want $\operatorname{argmin}_{\theta_0, \theta_1} J(\theta_0, \theta_1)$



Outline:

- Start with some θ_0, θ_1
- Keep changing θ_0, θ_1 to reduce $J(\theta_0, \theta_1)$
until we hopefully end up at minimum

Gradient descent

Repeat until convergence{

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \quad (\text{for } j = 0 \text{ and } j = 1)$$

}

cost function

α : Learning rate (step size)

$\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$: derivative (rate of change)

Gradient descent

Correct: simultaneous update

$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$

$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$

$$\theta_0 := \text{temp0}$$

$$\theta_1 := \text{temp1}$$

Incorrect:

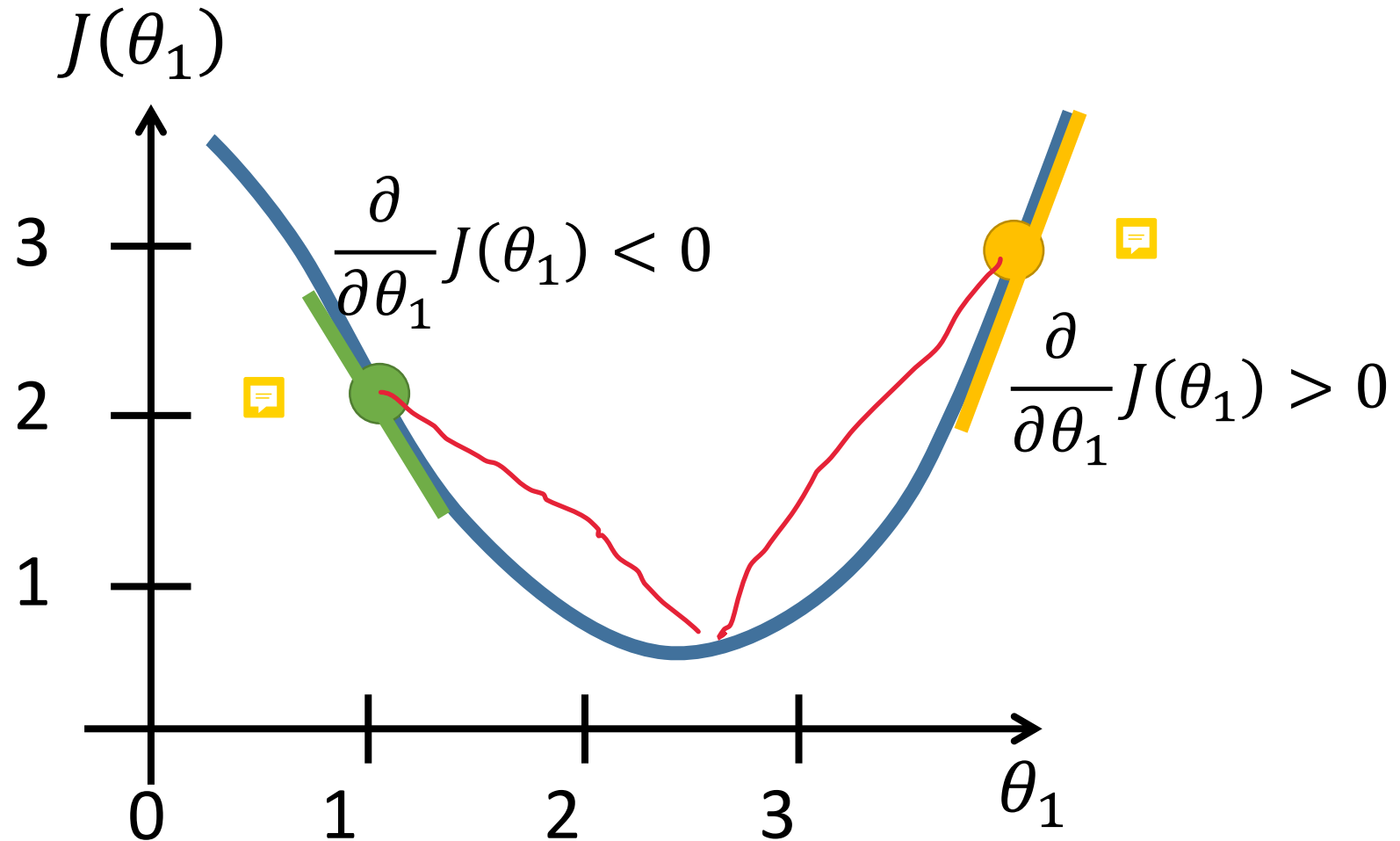
$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$

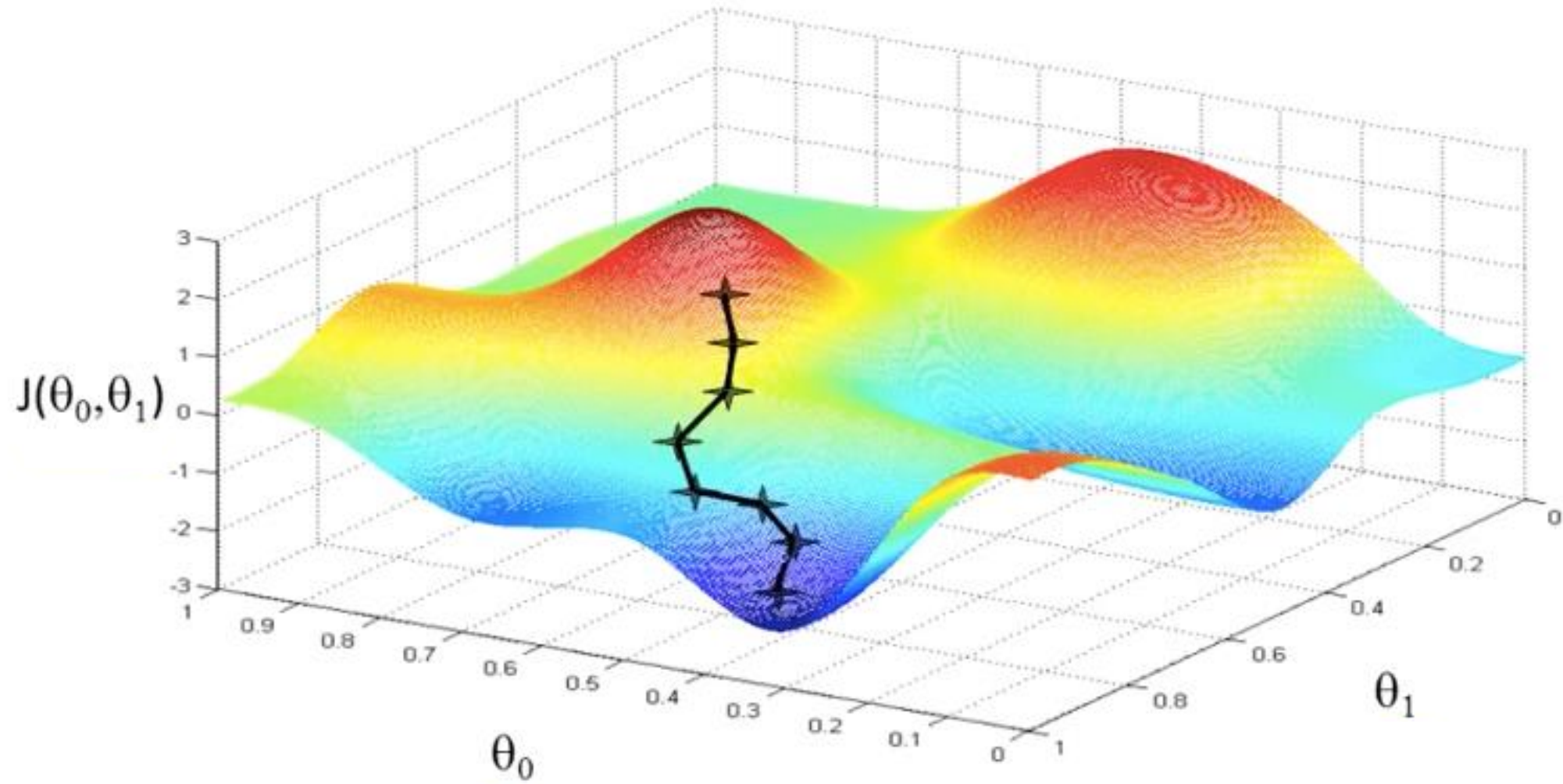
$$\theta_0 := \text{temp0}$$

$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$

$$\theta_1 := \text{temp1}$$

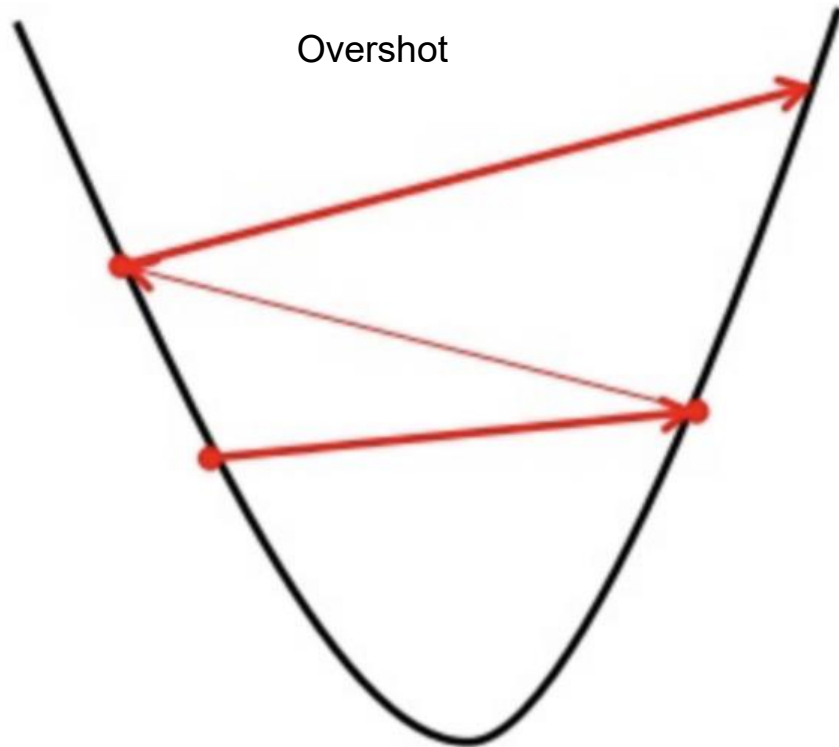
$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$



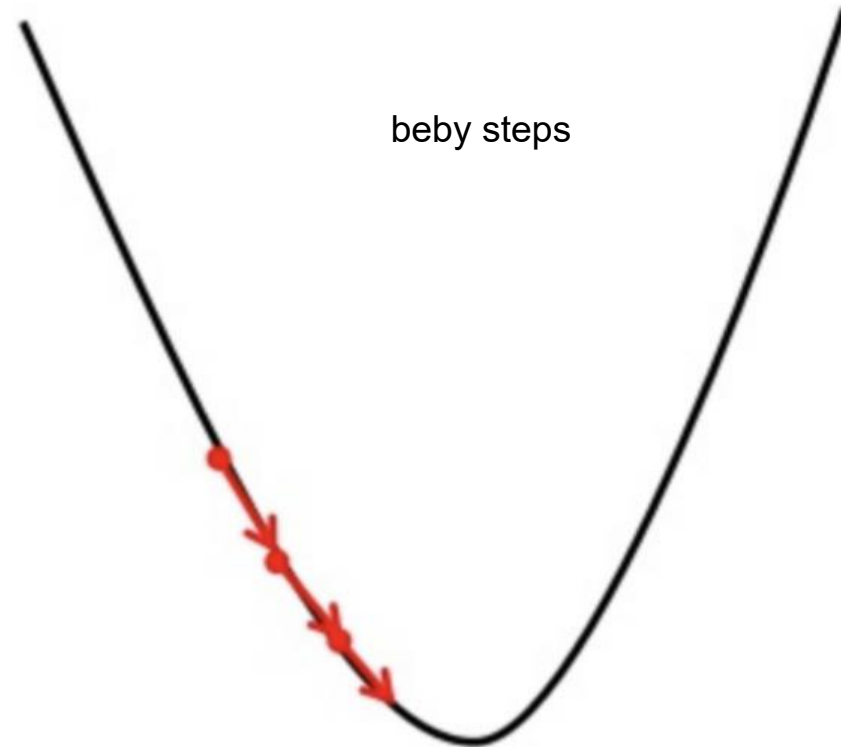


Learning rate

Big learning rate

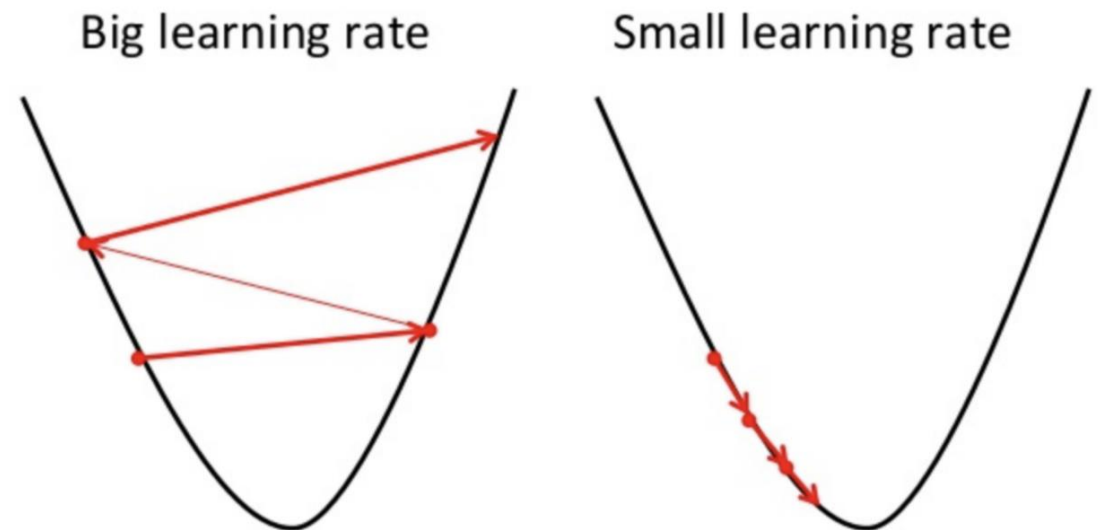


Small learning rate



Learning rate

- Learning rate gives the rate of speed where the gradient moves during gradient descent.
- Setting it too high would make your path instable
- too low values would make convergence slow.
- Put it to zero means your model isn't learning anything from the gradients.



Gradient descent for linear regression

Repeat until convergence{

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \quad (\text{for } j = 0 \text{ and } j = 1)$$

}

- Linear regression model

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Step by Step Algorithm

1- Choose initial values .Let $\theta_0 = 0$ and , $\theta_1 = 0$. Let L be our learning rate. It could be a small value like 0.01 for good accuracy.

2- Calculate the partial derivative of the Cost function with respect to θ_0 and θ_1 .

Computing partial derivative

$$\begin{aligned}\bullet \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) &= \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \\ &= \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x^{(i)} - y^{(i)})^2\end{aligned}$$

$$\bullet \underbrace{j = 0}: \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\bullet j = 1: \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Gradient descent for linear regression

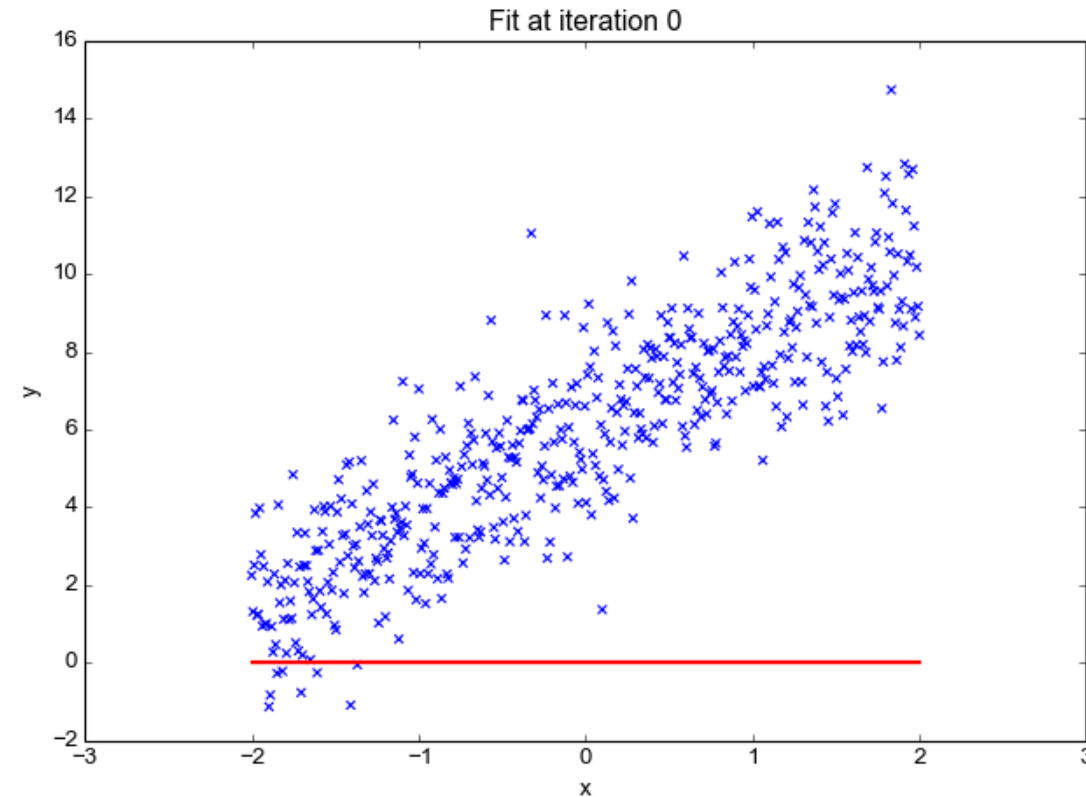
Repeat until convergence{

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

}

Update θ_0 and θ_1 simultaneously



Training dataset

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Batch gradient descent

- “Batch”: Each step of gradient descent uses all the training examples

Repeat until convergence{

m : Number of training examples

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

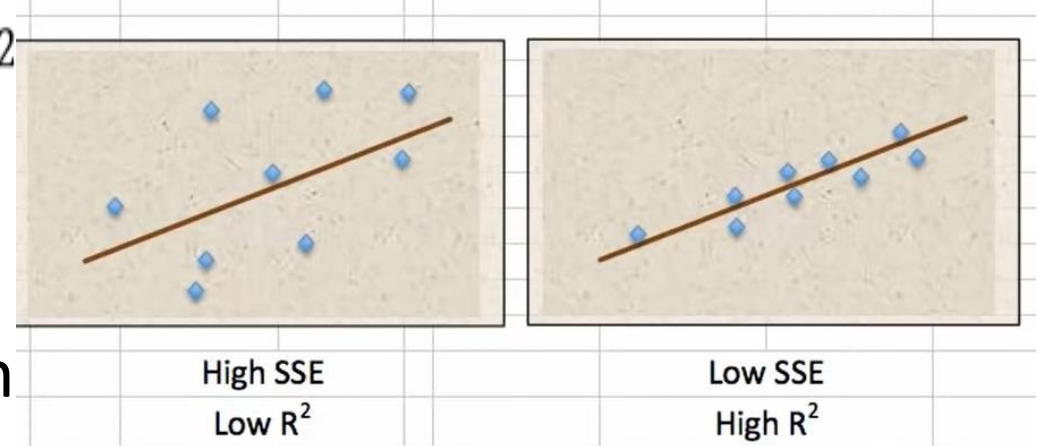
$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

}

Regression Evaluation metrics???

- Mean Absolute Error (MAE)
- Sum Squared Error (SSE) $\sum (Y_{actual} - Y_{predicted})^2$
- SSR $\sum (Y_{actual} - Y_{mean})^2$
- SST = SSE + SSR
- R Squared / Coefficient of Determination
- Adjusted R Square

$$MAE = \frac{1}{n} \sum_{j=1}^n |y_j - \hat{y}_j|$$



$$R - Square = 1 - \frac{\sum (Y_{actual} - Y_{predicted})^2}{\sum (Y_{actual} - Y_{mean})^2}$$

$$R^2 \text{ adjusted} = 1 - \frac{(1 - R^2)(N - 1)}{N - p - 1}$$



```
import numpy as np
```

```
def gradient_descent(  
    gradient, x, y, start, learn_rate=0.1, n_iter=50, tolerance=1e-06):  
    vector = start  
    for _ in range(n_iter):  
        diff = -learn_rate * np.array(gradient(x, y, vector))  
        if np.all(np.abs(diff) <= tolerance):  
            break  
        vector += diff  
    return vector
```

- `model = LinearRegression()`
- This statement creates the variable `model` as the instance of `LinearRegression`. You can provide several optional parameters to `LinearRegression`:
 - `fit_intercept` is a Boolean (True by default) that decides whether to calculate the intercept b_0 (True) or consider it equal to zero (False).
 - `normalize` is a Boolean (False by default) that decides whether to normalize the input variables (True) or not (False).
 - `copy_X` is a Boolean (True by default) that decides whether to copy (True) or overwrite the input variables (False).

Scikit Regression Lib.

Training the linear regression model:

```
from sklearn.linear_model import LinearRegression  
regressor = LinearRegression()  
regressor.fit(X_train, y_train)
```

Using the model for predicting:

```
y_pred = regressor.predict(X_test)
```

Calculate R square

```
r_sq = model.score(x, y)
```

Linear Regression() Methods

- `fit(X, y)` Fit linear model.
- `predict(X)` Predict using the linear model.
- `score(X, y)` Return the coefficient of determination of the prediction.

Scikit Regression Lib

Visualizing the results:

```
plt.scatter(X_train, y_train, color = 'red')  
plt.plot(X_train, regressor.predict(X_train), color = 'blue')  
plt.title('Title')  
plt.xlabel('x')  
plt.ylabel('y')  
plt.show()
```

Assignment 2: Apply the linear regression model to the auto-mpg dataset

- **Task 1(Practical):**
- **Objective:** In This Assignment, You Will Apply the Simple linear regression algorithm to predict city-cycle fuel consumption in miles per gallon(mpg) in terms of 3 multivalued discrete and 5 continuous attributes." (Quinlan, 1993)
- 1- Load the data from the file (`auto-mpg.data`) into the DataFrame. Set the names of columns according to the column definitions given in Data Description.
- 2- Data inspection. Display the first 5 rows of the dataset and use any relevant functions that can help you to understand the data. Use any necessary visualization techniques to inspect your data
- 3- **Prepare the data for regression.** Now you have to select the best 3 different independent variables to be used ***individually*** in your linear regression Model. Justify your reasoning
- 4-Using the pandas operators to prepare the feature variables `X` and the response `Y` for the fit. Note that `sklearn` expects data as arrays, so convert extracted columns into arrays. Split the data into `train` and `test` using `sklearn` `train\_test\_split` function.
- 5- Use the different regression learning models to predict the mpg values for the test dataset, run prediction on `X` from test part. Show the R-squared score of the prediction. Use plots to visualize the results.
- 6-Generate 5 random train/test splits to Fit the model and for each split and calculate the R-squared score for split then calculate the average R-squared score for the 5 runs.
- 7- Does the test scores meet the expectations you had in step 3. What conclusion can you make based on the results?
- 8- Build your own linear regression Model based on Gradient Decent optimization technique for one on models in step 3. Find the R-squared score for your model

Assignment 2

- **Task 2(Research):**
- Conduct a short research regarding the Adjusted R-Squared metric used for regression evaluation? How it differs from the regular R-squared value?